Code Start

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf
import numpy as np
from statsmodels.graphics.tsaplots import plot_pacf
from pandas import read_csv
from datetime import datetime
```

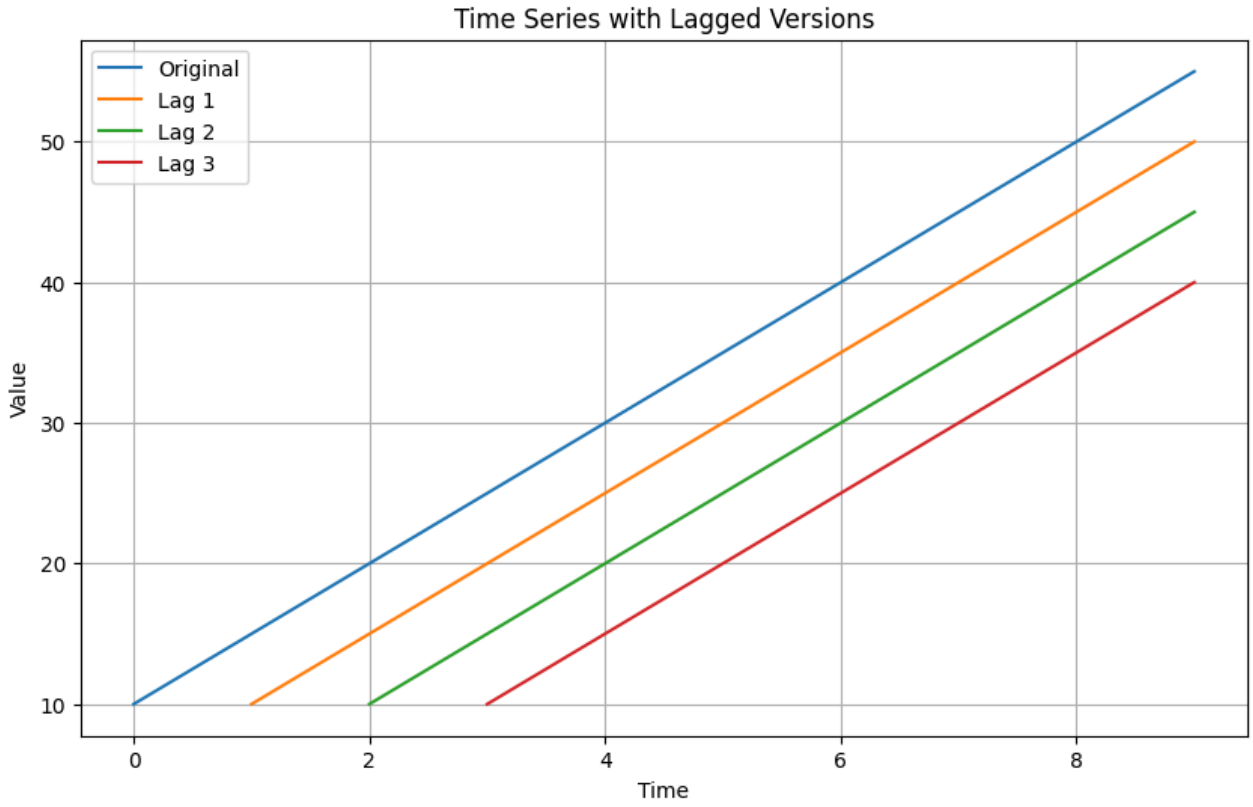
```
data = pd.Series([10, 15, 20, 25, 30, 35, 40, 45, 50, 55])

# Generate lagged versions
lag_1 = data.shift(1)
lag_2 = data.shift(2)
lag_3 = data.shift(3)
```

```
lag_1
```

	0
0	NaN
1	10.0
2	15.0
3	20.0
4	25.0
5	30.0
6	35.0
7	40.0
8	45.0
9	50.0

dtype: float64

```
plt.figure(figsize=(10, 6))
plt.plot(data, label='Original')
plt.plot(lag_1, label='Lag 1')
plt.plot(lag_2, label='Lag 2')
plt.plot(lag_3, label='Lag 3')
plt.xlabel('Time')
plt.ylabel('Value')
plt.title('Time Series with Lagged Versions')
plt.legend()
plt.grid(True)
plt.show()
```



Example

```
def check_stationarity(series):

    result = adfuller(series.values)

    print('ADF Statistic: %f' % result[0])
    print('p-value: %f' % result[1])
    print('Critical Values:')
    for key, value in result[4].items():
        print('\t%s: %.3f' % (key, value))

    if (result[1] <= 0.05) & (result[4]['5%'] > result[0]):
        print("\u001b[32mStationary\u001b[0m")
    else:
        print("\x1b[31mNon-stationary\x1b[0m")
```

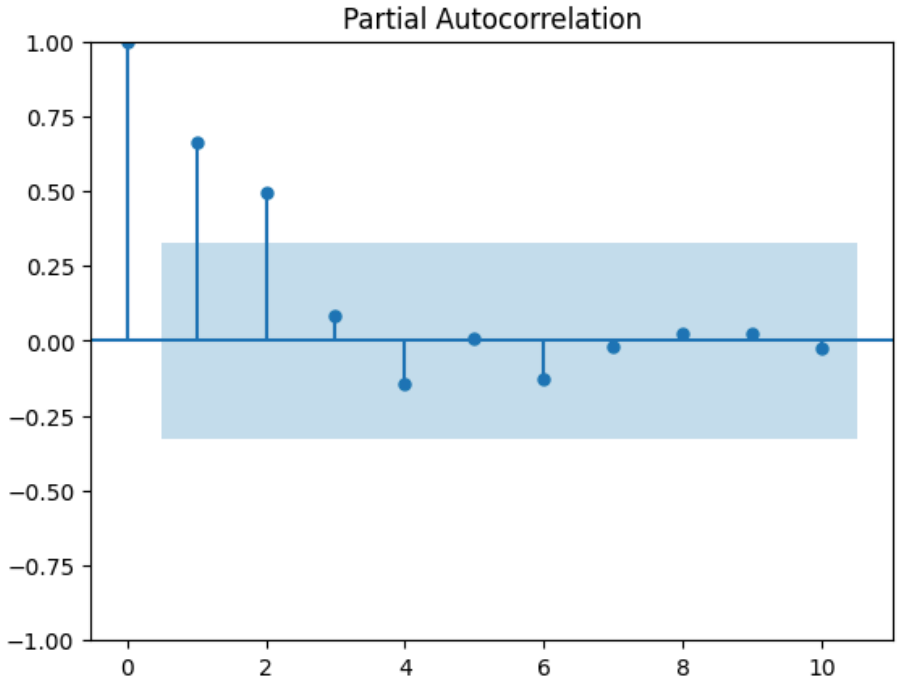
```
Start coding or generate with AI.
```

```
dateparse = lambda x: datetime.strptime('190'+x, '%Y-%m')
```

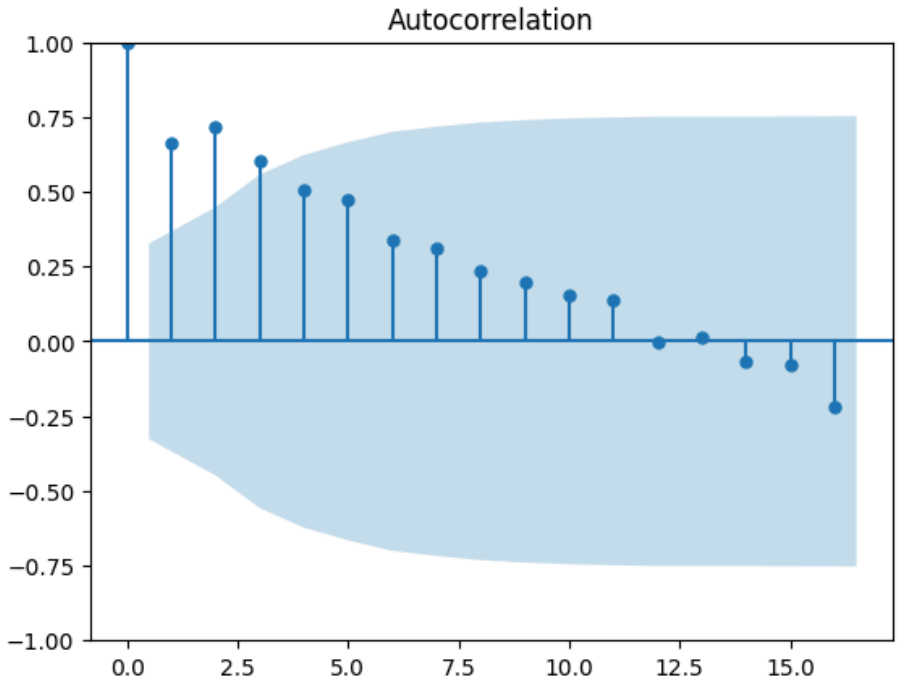
```
series_2 = read_csv('shampoo_sales.csv', parse_dates=[0], header=0, index_col=0, date_parser=datepars
```

```
/tmp/ipython-input-2627788734.py:1: FutureWarning: The argument 'date_parser' is deprecated and will b
  series_2 = read_csv('shampoo_sales.csv', parse_dates=[0], header=0, index_col=0, date_parser=datepar
```

```
plot_pacf(series_2, lags=10)
plt.show()
```




```
plot_acf(series_2)
plt.show()
```



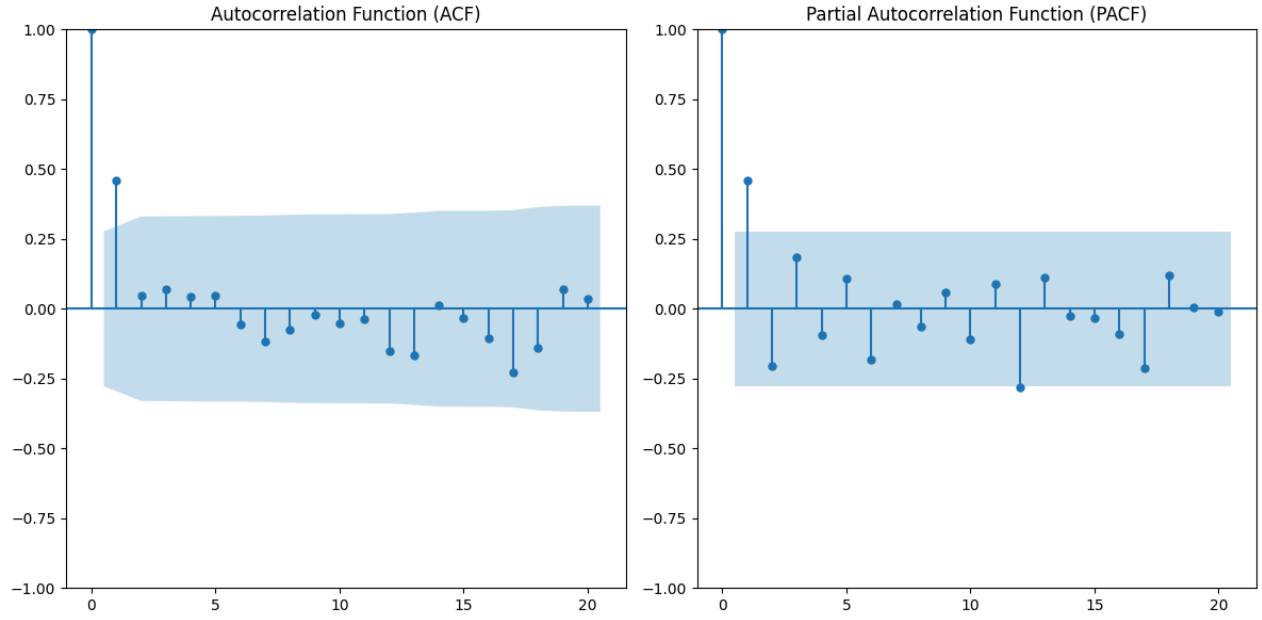
Start coding or generate with AI.

Start coding or generate with AI.

```
import pandas as pd
import numpy as np

# Parameters
np.random.seed(42)
n = 50  # Number of data points
mu = 0  # Mean of the series
theta = 0.7  # MA parameter
```

```
# Generate MA(1) time series data
errors = np.random.normal(loc=0, scale=1, size=n)
data = [mu + errors[0]]  # Initial value
for i in range(1, n):
    data.append(mu + theta * errors[i-1] + errors[i])

# Create pandas DataFrame
df = pd.DataFrame({'Time': pd.date_range(start='2024-01-01', periods=n, freq='D'),
                   'Data': data})
```

```
plt.figure(figsize=(12, 6))
plot_acf(df['Data'], lags=20, ax=plt.subplot(1, 2, 1))
plt.title('Autocorrelation Function (ACF)')
plot_pacf(df['Data'], lags=20, ax=plt.subplot(1, 2, 2))
plt.title('Partial Autocorrelation Function (PACF)')
plt.tight_layout()
plt.show()
```



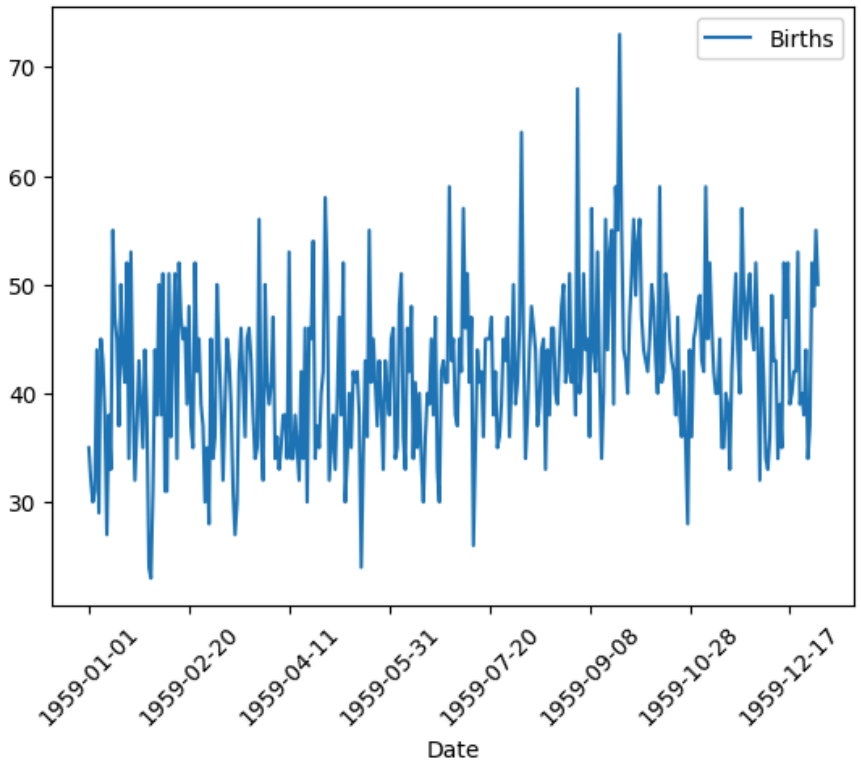

```
Start coding or generate with AI.
```

```
import pandas as pd
from datetime import datetime
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from pandas import read_csv
```

```
dateparse = lambda x: datetime.strptime(x, '%Y-%m-%d')
```

```
series = pd.read_csv('daily-total-female-births.csv',header=0, index_col=0)
```

```
series.plot(legend=True)
plt.xticks(rotation=45)  # Adjust the rotation angle as needed
plt.show()
```

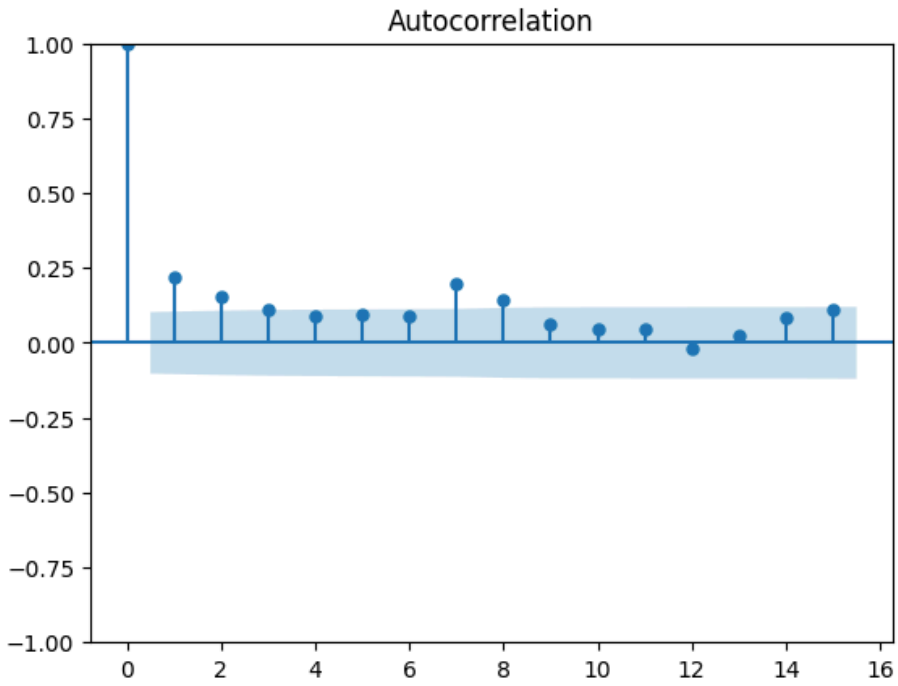


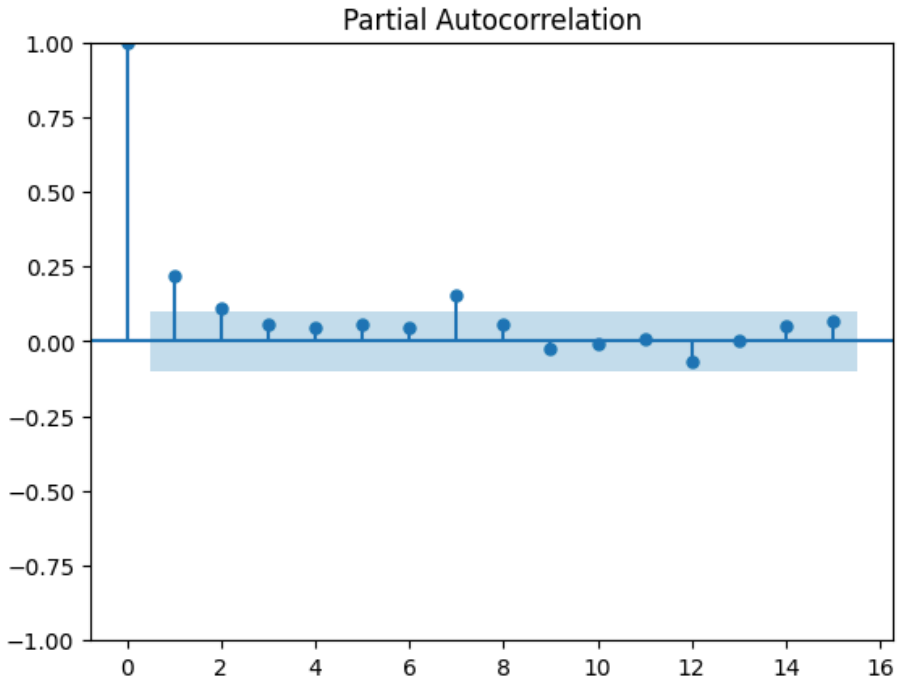

```
def perform_adf_test(series):
    result = adfuller(series)
    print('ADF Statistic: %f' % result[0])
    print('p-value: %f' % result[1])
```

```
result = adfuller(series)
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
```

```
ADF Statistic: -4.808291
p-value: 0.000052
Critical Values:
        1%: -3.449
        5%: -2.870
        10%: -2.571
```

```
plot_acf(series, lags=15)
plot_pacf(series, lags=15)
plt.show()
```





We can check AR(1), AR(2), AR(7)

```
ar_orders = [1, 2,7]
fitted_model_dict = {}

for idx, ar_order in enumerate(ar_orders):

    #create AR(p) model
    ar_model = ARIMA(series, order=(ar_order,0,0))
    ar_model_fit = ar_model.fit()
    fitted_model_dict[ar_order] = ar_model_fit
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
```

```
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
```

```
for ar_order in ar_orders:
    print('AIC for AR(%s): %s'%(ar_order, fitted_model_dict[ar_order].aic))
```

```
AIC for AR(1): 2479.080627834601
AIC for AR(2): 2476.363657107182
AIC for AR(7): 2472.778262629018
```

```
for ar_order in ar_orders:
    print('BIC for AR(%s): %s'%(ar_order, fitted_model_dict[ar_order].bic))
```

```
BIC for AR(1): 2490.7803198953484
BIC for AR(2): 2491.963246521512
BIC for AR(7): 2507.8773388112604
```

If we treat this as a ARMA process

```
from itertools import product

p_values = range(1, 3)
q_values = range(1, 3)

fitted_model_dict = {}

for p, q in product(p_values, q_values):
    # Create ARMA(p,q) model
    arma_model = ARIMA(series, order=(p, 0, q))
    arma_model_fit = arma_model.fit()

    # Store AIC and BIC scores
    aic_score = arma_model_fit.aic
    bic_score = arma_model_fit.bic

    fitted_model_dict[(p, q)] = {'AIC': aic_score, 'BIC': bic_score}
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
```

```
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
```

```
for (p, q), scores in fitted_model_dict.items():
    print(f'ARMA({p},{q}) - AIC: {scores["AIC"]}, BIC: {scores["BIC"]}')
```

```
ARMA(1,1) - AIC: 2468.905798033242, BIC: 2484.5053874475716
ARMA(1,2) - AIC: 2466.724037901049, BIC: 2486.2235246689615
ARMA(2,1) - AIC: 2466.283266457811, BIC: 2485.7827532257234
ARMA(2,2) - AIC: 2467.6978578156804, BIC: 2491.0972419371756
```

```
def parser(s):
    return datetime.strptime(s, '%Y-%m-%d')
```

```
series = pd.read_csv('catfish.csv', parse_dates=[0], index_col=0, date_parser=parser)
series = series.asfreq(pd.infer_freq(series.index))
series = series.loc[datetime(2004,1,1):]
series = series.diff().diff().dropna()
```

```
/tmp/ipython-input-4154342099.py:1: FutureWarning: The argument 'date_parser' is deprecated and will b
  series = pd.read_csv('catfish.csv', parse_dates=[0], index_col=0, date_parser=parser)
```

```
def perform_adf_test(series):
    result = adfuller(series)
    print('ADF Statistic: %f' % result[0])
    print('p-value: %f' % result[1])
```

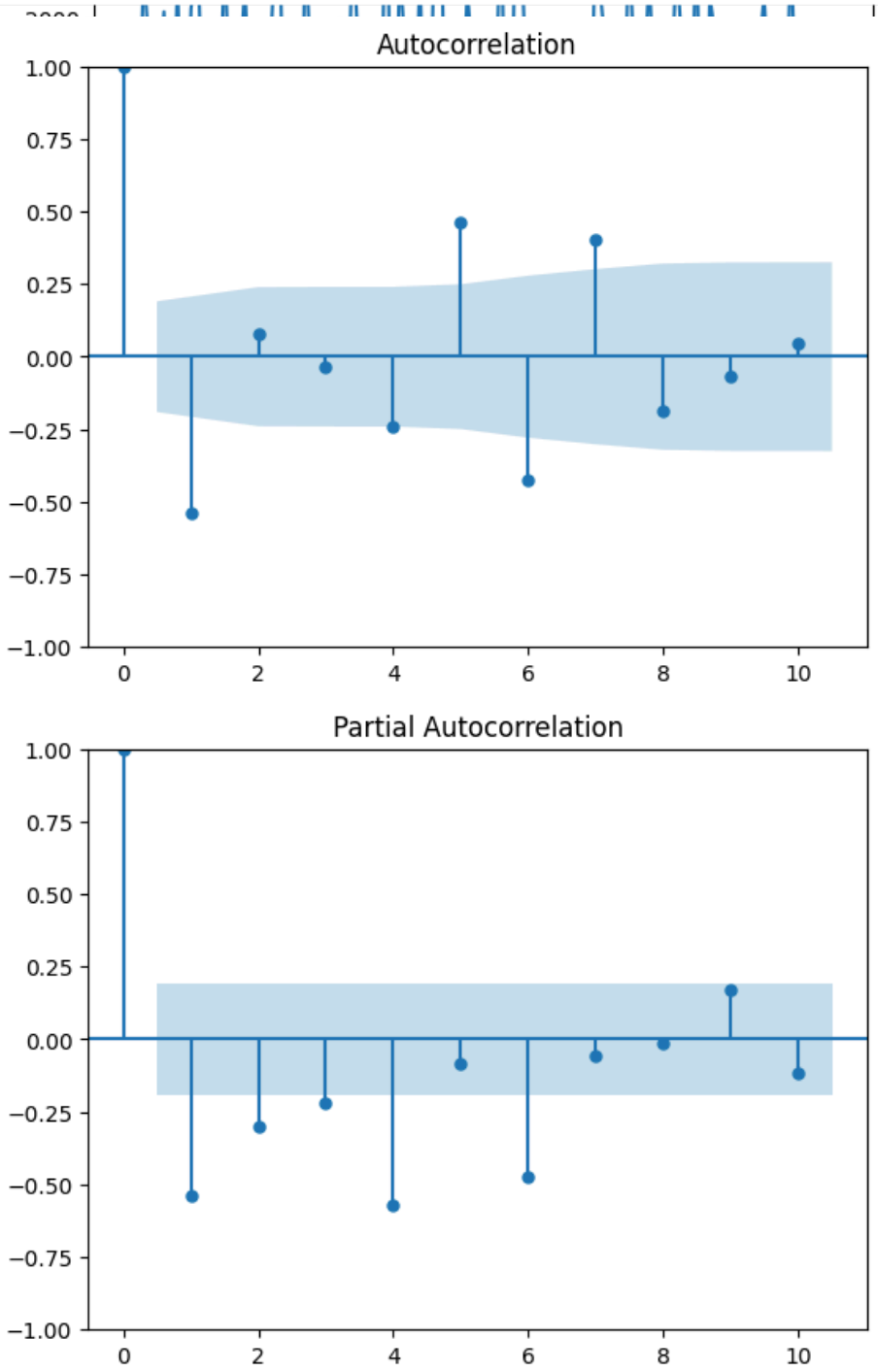
```
perform_adf_test(series)
```

```
ADF Statistic: -7.162321
p-value: 0.000000
```

```
plt.plot(series)
```

```
[<matplotlib.lines.Line2D at 0x7c45b0945ac0>]
```

```
plot_acf(series, lags=10)
plot_pacf(series, lags=10)

plt.show()
```




```
ar_orders = [1, 4, 6, 10]
fitted_model_dict = {}

for idx, ar_order in enumerate(ar_orders):

    #create AR(p) model
    ar_model = ARIMA(series, order=(ar_order,0,0))
    ar_model_fit = ar_model.fit()
    fitted_model_dict[ar_order] = ar_model_fit
```

```
for ar_order in ar_orders:
    print('AIC for AR(%s): %s'%(ar_order, fitted_model_dict[ar_order].aic))
```

```
AIC for AR(1): 1980.860621744531
AIC for AR(4): 1927.609985266023
AIC for AR(6): 1899.6497440836226
AIC for AR(10): 1902.3765450081733
```

```
#BIC comparison
for ar_order in ar_orders:
    print('BIC for AR(%s): %s'%(ar_order, fitted_model_dict[ar_order].bic))
```

```
BIC for AR(1): 1988.8509390268673
BIC for AR(4): 1943.5906198306952
BIC for AR(6): 1920.9572568365193
BIC for AR(10): 1934.337814137518
```

```
import pandas as pd
import matplotlib.pyplot as plt
import itertools
import warnings

from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
```

```
!pip install pmdarima
```

```
Requirement already satisfied: pmdarima in /usr/local/lib/python3.12/dist-packages (2.0.4)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.12/dist-packages (from pmdarima)
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.12/dist-packages (from pmdarima
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.12/dist-packages (from pmdarima)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.12/dist-packages (from pmd
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.12/dist-packages (from pmdarima)
Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.12/dist-packages (from pm
Requirement already satisfied: urllib3 in /usr/local/lib/python3.12/dist-packages (from pmdarima) (2.5
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.12/dist-packages (from pmdari
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from s
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from statsmode
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateut
```

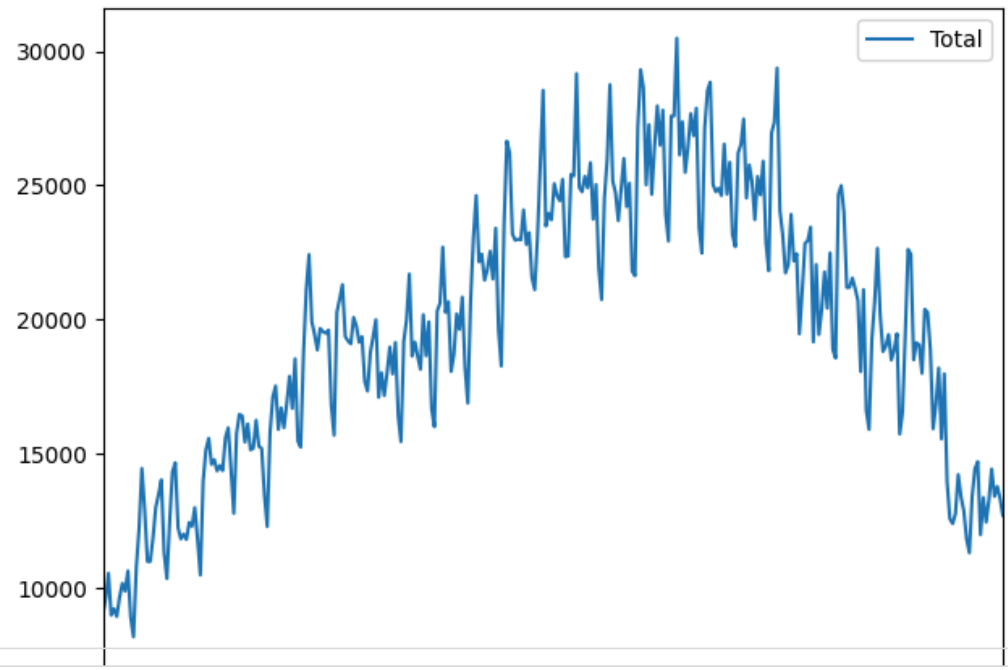
```
Start coding or generate with AI.
```

```
df_2=pd.read_csv('catfish.csv')
```

```
df_2['Date'] = pd.to_datetime(df_2['Date'])
```

```
df_2.set_index('Date', inplace=True)
```

```
df_2['Total'].plot(legend=True)
plt.tight_layout()
plt.show()
```




```
df_2
```

Date	Total
1986-01-01	9034
1986-02-01	9596
1986-03-01	10558
1986-04-01	9002
1986-05-01	9239
...	...
2012-08-01	14442
2012-09-01	13422
2012-10-01	13795
2012-11-01	13352
2012-12-01	12716

324 rows × 1 columns

Next steps: Generate code with df_2 View recommended plots New interactive sheet

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
# train_size = int(len(df_2) * 0.8)  # 80% of data for training
# train_data_2, test_data_2 = df_2[:train_size], df_2[train_size:]
```

```
train_size = int(len(df_2) * 0.80)
train_data_2, test_data_2 = df_2[0:train_size], df_2[train_size:]
```

```
start_2=len(train_data_2)
end_2=len(train_data_2)+ len(test_data_2)-1
```

```
model_4=ARIMA(train_data_2,order=(1,0,1))
model_4=model_4.fit()
model_4.summary()
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No freque
  self._init_dates(dates, freq)
```

SARIMAX Results

Dep. Variable:	Total	**No. Observations:**	259
Model:	ARIMA(1, 0, 1)	**Log Likelihood**	-2318.444
Date:	Mon, 15 Sep 2025	**AIC**	4644.888
Time:	16:49:22	**BIC**	4659.115
Sample:	01-01-1986 - 07-01-2007	**HQIC**	4650.608
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
const	2.016e+04	5432.597	3.711	0.000	9513.102	3.08e+04
ar.L1	0.9963	0.006	177.199	0.000	0.985	1.007
ma.L1	-0.7030	0.046	-15.245	0.000	-0.793	-0.613
sigma2	3.445e+06	20.054	1.72e+05	0.000	3.45e+06	3.45e+06

Ljung-Box (L1) (Q):	20.16	**Jarque-Bera (JB):**	4.52
Prob(Q):	0.00	**Prob(JB):**	0.10
Heteroskedasticity (H):	1.80	**Skew:**	0.32
Prob(H) (two-sided):	0.01	**Kurtosis:**	2.87

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 8.01e+24. Standard errors may be unstable.

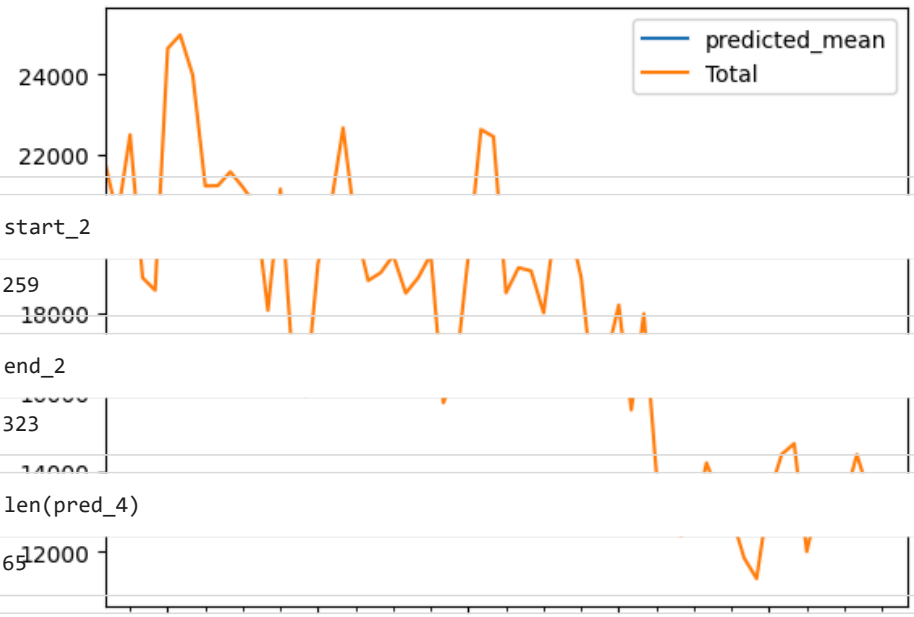
```
pred_4=model_4.predict(start=start_2,end=end_2,type='levels')
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/statespace/representation.py:374: FutureWarnin
  warnings.warn(msg, FutureWarning)
```

```
pred_4.index=df_2.index[start_2:end_2+1]
```

```
pred_4.plot(legend=True)
test_data_2['Total'].plot(legend=True)
```

```
<Axes: xlabel='Date'>
```




```
start_2
259
end_2
323
len(pred_4)
65
```

```
from sklearn.metrics import mean_absolute_error
```

```
mae=mean_absolute_error(pred_4,test_data_2['Total'])
print(mae)
```

```
3844.5009324866705
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

Grid Search

```
Start coding or generate with AI.
```

```
import pandas as pd
from statsmodels.tsa.stattools import adfuller

def check_stationarity(series, max_diff_order=2, alpha=0.05):
    """
    Check stationarity of a time series using the Augmented Dickey-Fuller (ADF) test.
    If the series is not stationary, apply differencing recursively until stationarity is achieved.

    Parameters:
    - series: pandas Series, the time series data
    - max_diff_order: int, maximum order of differencing to be applied (default: 2)
    - alpha: float, significance level for ADF test (default: 0.05)

    Returns:
    - stationary_series: pandas Series, the stationary time series
    """
    # Perform ADF test
    adf_result = adfuller(series)
    p_value = adf_result[1]

    # If the series is already stationary, return the original series
    if p_value < alpha:
        print("Original series is stationary (p-value: {:.4f})".format(p_value))
        return series
```

```
    # If the series is not stationary, apply differencing
    print("Original series is not stationary (p-value: {:.4f}), applying differencing...".format(p_va
    for diff_order in range(1, max_diff_order + 1):
        differenced_series = series.diff(diff_order).dropna()
        adf_result_diff = adfuller(differenced_series)
        p_value_diff = adf_result_diff[1]

        # Check stationarity after differencing
        if p_value_diff < alpha:
            print("Series after {} order differencing is stationary (p-value: {:.4f})".format(diff_or
            return differenced_series

    # If maximum differencing orders reached and still not stationary, return None
    print("Maximum differencing orders reached. Unable to achieve stationarity.")
    return None

# Example usage:
# Assuming 'time_series' is your pandas Series containing the time series data
# stationary_series = check_stationarity(time_series)
```

```
def difference(dataset, interval=1):
 diff = list()
 for i in range(interval, len(dataset)):
     value = dataset[i] - dataset[i - interval]
     diff.append(value)
 return Series(diff)
```

```
stat=check_stationarity(df_2['Total'])
```

```
Original series is not stationary (p-value: 0.4887), applying differencing...
Series after 1 order differencing is stationary (p-value: 0.0004)
```

```
def evaluate_arima_model(X, arima_order):
 # prepare training dataset
 train_size = int(len(X) * 0.80)
 train, test = X[0:train_size], X[train_size:]
 history = [x for x in train]
 predictions = list()
 for t in range(len(test)):
     model = ARIMA(history, order=arima_order)
     model_fit = model.fit()
     yhat = model_fit.forecast()[0]
     predictions.append(yhat)
     history.append(test[t])
 # # calculate out of sample error
 rmse = sqrt(mean_squared_error(test, predictions))
 return rmse,predictions
```

```
def evaluate_models(dataset, p_values, d_values, q_values):
    dataset = dataset.astype('float32')
    best_score, best_cfg = float("inf"), None
    for p in p_values:
        for d in d_values:
            for q in q_values:
                order = (p,d,q)
                try:
                    rmse,predictions = evaluate_arima_model(dataset, order)
                    if rmse < best_score:
                        best_score, best_cfg = rmse, order
                    print('ARIMA%s RMSE=%.3f' % (order,rmse))
```

```
                except:
                    continue
    print('Best ARIMA%s RMSE=%.3f' % (best_cfg, best_score))
    return predictions
```

```
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
import numpy as np
import warnings

def evaluate_arima_model(X, arima_order):
    train_size = int(len(X) * 0.8)
    train, test = X[:train_size], X[train_size:]

    model = ARIMA(train, order=arima_order)
    model_fit = model.fit()

    forecast = model_fit.forecast(steps=len(test))

    rmse = mean_squared_error(test, forecast, squared=False)
    return rmse, forecast

def evaluate_models(dataset, p_values, d_values, q_values):
    best_score, best_cfg, best_predictions = float("inf"), None, None

    for p in p_values:
        for d in d_values:
            for q in q_values:
                order = (p,d,q)
                try:
                    rmse, predictions = evaluate_arima_model(dataset, order)
                    print(f"ARIMA{order} RMSE={rmse:.3f}")
                    if rmse < best_score:
                        best_score, best_cfg, best_predictions = rmse, order, predictions
                except:
                    continue

    if best_cfg is not None:
        print(f"Best ARIMA{best_cfg} RMSE={best_score:.3f}")
    else:
        print("No valid ARIMA model found.")

    return best_predictions
```

```
pip install darts
```

```
Collecting darts
  Downloading darts-0.37.1-py3-none-any.whl.metadata (60 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.4/60.4 kB 2.4 MB/s eta 0:00:00
Requirement already satisfied: holidays>=0.11.1 in /usr/local/lib/python3.12/dist-packages (from dart
Requirement already satisfied: joblib>=0.16.0 in /usr/local/lib/python3.12/dist-packages (from darts
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from da
Requirement already satisfied: narwhals>=1.25.1 in /usr/local/lib/python3.12/dist-packages (from dart
Collecting nfoursid>=1.0.0 (from darts)
  Downloading nfoursid-1.0.2-py3-none-any.whl.metadata (1.9 kB)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from darts)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-packages (from darts)
Collecting pyod>=0.9.5 (from darts)
  Downloading pyod-2.0.5-py3-none-any.whl.metadata (46 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.3/46.3 kB 2.0 MB/s eta 0:00:00
Requirement already satisfied: requests>=2.22.0 in /usr/local/lib/python3.12/dist-packages (from dart
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from d
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.12/dist-packages (from darts)
Requirement already satisfied: shap>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from darts)
Collecting statsforecast>=1.4 (from darts)
  Downloading statsforecast-2.0.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadat
Requirement already satisfied: statsmodels>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from d
Requirement already satisfied: tqdm>=4.60.0 in /usr/local/lib/python3.12/dist-packages (from darts)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from da
Requirement already satisfied: xarray>=0.17.0 in /usr/local/lib/python3.12/dist-packages (from darts
Requirement already satisfied: xgboost>=2.1.4 in /usr/local/lib/python3.12/dist-packages (from darts
Collecting pytorch-lightning<2.5.3,>=1.5.0 (from darts)
  Downloading pytorch_lightning-2.5.2-py3-none-any.whl.metadata (21 kB)
Collecting tensorboardX>=2.1 (from darts)
  Downloading tensorboardx-2.6.4-py3-none-any.whl.metadata (6.2 kB)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from darts)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from holid
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matp
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotl
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from mat
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from mat
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matpl
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matp
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas
Requirement already satisfied: numba>=0.51 in /usr/local/lib/python3.12/dist-packages (from pyod>=0.9
Requirement already satisfied: PyYAML>=5.4 in /usr/local/lib/python3.12/dist-packages (from pytorch-l
Requirement already satisfied: fsspec>=2022.5.0 in /usr/local/lib/python3.12/dist-packages (from fssp
Collecting torchmetrics>=0.7.0 (from pytorch-lightning<2.5.3,>=1.5.0->darts)
  Downloading torchmetrics-1.8.2-py3-none-any.whl.metadata (22 kB)
Collecting lightning-utilities>=0.10.0 (from pytorch-lightning<2.5.3,>=1.5.0->darts)
  Downloading lightning_utilities-0.15.2-py3-none-any.whl.metadata (5.7 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from re
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from re
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap>=0
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap>=0.4
Collecting coreforecast>=0.0.12 (from statsforecast>=1.4->darts)
  Downloading coreforecast-0.0.16-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata
Collecting scipy>=1.3.2 (from darts)
  Downloading scipy-1.15.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.0/62.0 kB 3.6 MB/s eta 0:00:00
Collecting fugue>=0.8.1 (from statsforecast>=1.4->darts)
  Downloading fugue-0.9.1-py3-none-any.whl.metadata (18 kB)
Collecting utilsforecast>=0.1.4 (from statsforecast>=1.4->darts)
  Downloading utilsforecast-0.2.12-py3-none-any.whl.metadata (7.6 kB)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from statsmod
Requirement already satisfied: protobuf>=3.20 in /usr/local/lib/python3.12/dist-packages (from tenso
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0-
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-pack
```

AutoARIMA

```
import itertools
from statsmodels.tsa.arima.model import ARIMA
```

```
p = d = q = range(0, 5)
best_aic, best_order, best_model = float("inf"), None, None

for order in itertools.product(p, d, q):
    try:
        model = ARIMA(train_data_2, order=order)
        model_fit = model.fit()
        if model_fit.aic < best_aic:
            best_aic, best_order, best_model = model_fit.aic, order, model_fit
    except:
        continue

print("Best ARIMA order:", best_order, "AIC:", best_aic)
print(best_model.summary())
```

```
Best ARIMA order: (4, 1, 4) AIC: 4450.468550231405
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  259
Model:                 ARIMA(4, 1, 4)   Log Likelihood               -2216.234
Date:                Mon, 15 Sep 2025   AIC                           4450.469
Time:                        17:24:00   BIC                           4482.445
Sample:                    01-01-1986   HQIC                          4463.327
                         - 07-01-2007
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.7427      0.022    -33.825      0.000      -0.786      -0.700
ar.L2         -0.2566      0.030     -8.459      0.000      -0.316      -0.197
ar.L3         -0.6996      0.027    -26.026      0.000      -0.752      -0.647
ar.L4         -0.9698      0.018    -53.508      0.000      -1.005      -0.934
ma.L1          0.6566      0.103      6.356      0.000       0.454       0.859
ma.L2          0.0060      0.058      0.104      0.917      -0.108       0.120
ma.L3         -0.5447      0.070     -7.747      0.000      -0.683      -0.407
ma.L4         -0.9272      0.104     -8.948      0.000      -1.130      -0.724
sigma2      2.292e+06   3.11e-08   7.36e+13      0.000    2.29e+06    2.29e+06
===================================================================================
Ljung-Box (L1) (Q):                   2.14   Jarque-Bera (JB):                 1.16
Prob(Q):                              0.14   Prob(JB):                         0.56
Heteroskedasticity (H):               1.49   Skew:                             0.16
Prob(H) (two-sided):                  0.07   Kurtosis:                         2.92
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 1.16e+29. Standard errors ma
```

```
model_3=ARIMA(train_data_2,order=(4,1,3))
model_3=model_3.fit()
model_3.summary()
```

```
Collecting numpy>=1.19.0 (from darts)
  Downloading numpy-2.0.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.9/60.9 kB 4.2 MB/s eta 0:00:00
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateu
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sy
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiol
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aic
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from a
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiol
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aic
Requirement already satisfied: pyarrow>=6.0.1 in /usr/local/lib/python3.12/dist-packages (from triad
Collecting fs (from triad>=0.9.7->fugue>=0.8.1->statsforecast>=1.4->darts)
  Downloading fs-2.4.16-py2.py3-none-any.whl.metadata (6.3 kB)
Collecting appdirs~=1.4.3 (from fs->triad>=0.9.7->fugue>=0.8.1->statsforecast>=1.4->darts)
  Downloading appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Downloading darts-0.37.1-py3-none-any.whl (1.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.1/1.1 MB 9.2 MB/s eta 0:00:00
Downloading nfoursid-1.0.2-py3-none-any.whl (18 kB)
Downloading pyod-2.0.5-py3-none-any.whl (200 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 200.6/200.6 kB 9.7 MB/s eta 0:00:00
Downloading pytorch_lightning-2.5.5-py3-none-any.whl (825 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 825.4/825.4 kB 17.9 MB/s eta 0:00:00
Downloading statsforecast-2.0.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (344 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 344.8/344.8 kB 10.5 MB/s eta 0:00:00
Downloading scipy-1.15.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (37.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37.3/37.3 MB 4.9 MB/s eta 0:00:00
Downloading tensorboardx-2.6.4-py3-none-any.whl (87 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 87.2/87.2 kB 3.5 MB/s eta 0:00:00
Downloading coreforecast-0.0.16-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (287 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 287.4/287.4 kB 10.3 MB/s eta 0:00:00
Downloading fugue-0.9.1-py3-none-any.whl (278 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.2/278.2 kB 15.1 MB/s eta 0:00:00
Downloading adagio-0.2.6-py3-none-any.whl (19 kB)
Downloading triad-0.9.8-py3-none-any.whl (62 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.3/62.3 kB 3.9 MB/s eta 0:00:00
Downloading fs-2.4.16-py2.py3-none-any.whl (135 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 135.3/135.3 kB 8.2 MB/s eta 0:00:00
Downloading appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Installing collected packages: appdirs, numpy, lightning-utilities, fs, tensorboardX, scipy, corefore
  Attempting uninstall: numpy
    Found existing installation: numpy 2.3.3
    Uninstalling numpy-2.3.3:
      Successfully uninstalled numpy-2.3.3
  Attempting uninstall: scipy
    Found existing installation: scipy 1.16.2
    Uninstalling scipy-1.16.2:
      Successfully uninstalled scipy-1.16.2
ERROR: pip's dependency resolver does not currently take into account all the packages that are insta
```

```
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 2.3.2 which is incompatible.
dask-cudf-cu12 25.6.0 requires pandas<2.2.4dev0,>=2.0, but you have pandas 2.3.2 which is incompatib
cudf-cu12 25.6.0 requires pandas<2.2.4dev0,>=2.0, but you have pandas 2.3.2 which is incompatible.
Successfully installed adagio-0.2.6 appdirs-1.4.4 coreforecast-0.0.16 darts-0.37.1 fs-2.4.16 fugue-0
WARNING: The following packages were previously imported in this runtime:
  [numpy,scipy]
You must restart the runtime in order to use newly installed versions.
```

RESTART SESSION

SARIMAX Results

Dep. Variable:	Total	**No. Observations:**	259
Model:	ARIMA(4, 1, 3)	**Log Likelihood**	-2231.495
Date:	Mon, 15 Sep 2025	**AIC**	4478.991
Time:	17:24:06	**BIC**	4507.414
Sample:	01-01-1986	**HQIC**	4490.420
	- 07-01-2007		

Covariance Type: opg

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	-1.0314	0.083	-12.427	0.000	-1.194	-0.869
ar.L2	-0.3177	0.146	-2.172	0.030	-0.604	-0.031
ar.L3	-0.2221	0.135	-1.649	0.099	-0.486	0.042
ar.L4	-0.5288	0.071	-7.495	0.000	-0.667	-0.391
ma.L1	0.9692	0.091	10.633	0.000	0.791	1.148
ma.L2	-0.2428	0.151	-1.608	0.108	-0.539	0.053
ma.L3	-0.6656	0.096	-6.967	0.000	-0.853	-0.478
sigma2	2.131e+06	1.17e-08	1.82e+14	0.000	2.13e+06	2.13e+06

Ljung-Box (L1) (Q):	0.73	**Jarque-Bera (JB):**	3.49
Prob(Q):	0.39	**Prob(JB):**	0.17
Heteroskedasticity (H):	1.42	**Skew:**	0.27
Prob(H) (two-sided):	0.11	**Kurtosis:**	2.82

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 9.22e+29. Standard errors may be unstable.

```
pred_3=model_3.predict(start=start_2,end=end_2,type='levels')
```

```
pred_3
```

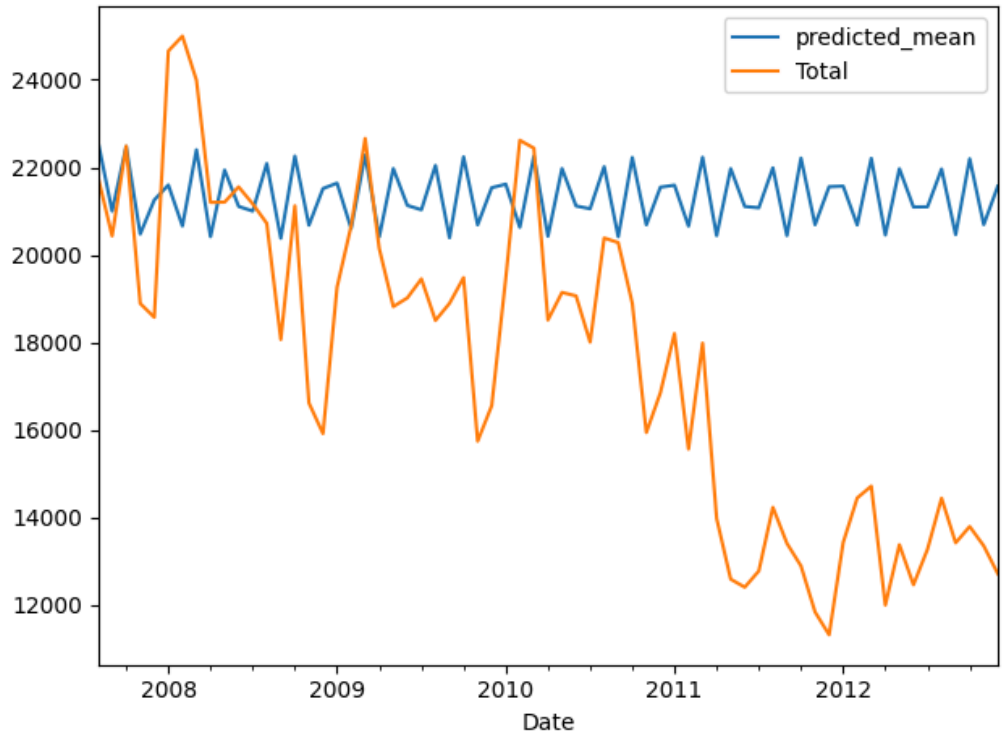
	predicted_mean
2007-08-01	22583.714610
2007-09-01	20998.738922
2007-10-01	22474.260926
2007-11-01	20475.740344
2007-12-01	21254.047157
...	...
2012-08-01	21961.606291
2012-09-01	20463.250518
2012-10-01	22195.474620
2012-11-01	20690.875321
2012-12-01	21568.606658

65 rows × 1 columns

dtype: float64

```
pred_3.index=df_2.index[start_2:end_2+1]
```

```
pred_3.plot(legend=True)
test_data_2['Total'].plot(legend=True)
plt.tight_layout()
```




```
from sklearn.metrics import mean_absolute_error
```

```
from math import sqrt
rmse=sqrt(mean_squared_error(pred_3,test_data_2['Total']))
print(rmse)
```

```
5182.459131057669
```

```
test_data_2['Total'].mean()
```

```
np.float64(17581.33846153846)
```

```
model_future=ARIMA(df_2['Total'],order=(4,1,3))
model_future=model_future.fit()
```

```
df_2.tail()
```

Date	Total
2012-08-01	14442
2012-09-01	13422
2012-10-01	13795
2012-11-01	13352
2012-12-01	12716

```
index_future=pd.date_range(start='2012-12-01',end='2013-08-01',freq='M')
```

```
index_future
```

```
DatetimeIndex(['2012-12-31', '2013-01-31', '2013-02-28', '2013-03-31',
               '2013-04-30', '2013-05-31', '2013-06-30', '2013-07-31'],
              dtype='datetime64[ns]', freq='ME')
```

```
pred_future=model_future.predict(start=len(df_2),end=len(df_2)+7,type='level')
```

```
pred_future
```

	predicted_mean
2013-01-01	12890.686883
2013-02-01	12496.383686
2013-03-01	14025.196727
2013-04-01	12807.877930
2013-05-01	13561.920658
2013-06-01	13067.019951
2013-07-01	12820.840809
2013-08-01	13709.794332

dtype: float64

```
pred_future.index=index_future
```

```
pred_future
```

	predicted_mean
2012-12-31	12890.686883
2013-01-31	12496.383686
2013-02-28	14025.196727
2013-03-31	12807.877930
2013-04-30	13561.920658
2013-05-31	13067.019951
2013-06-30	12820.840809
2013-07-31	13709.794332

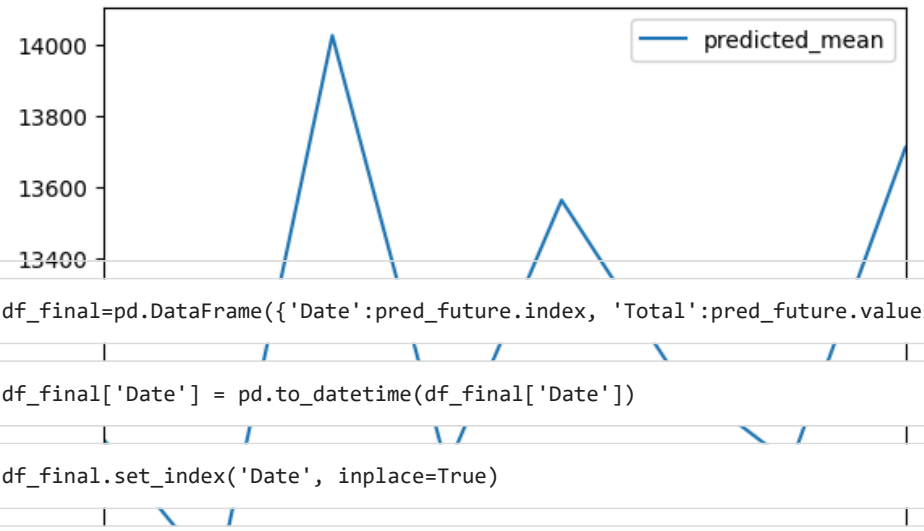
dtype: float64

```
len(df_2)
```

```
324
```

```
pred_future.plot(legend=True)
```

```
<Axes: >
```

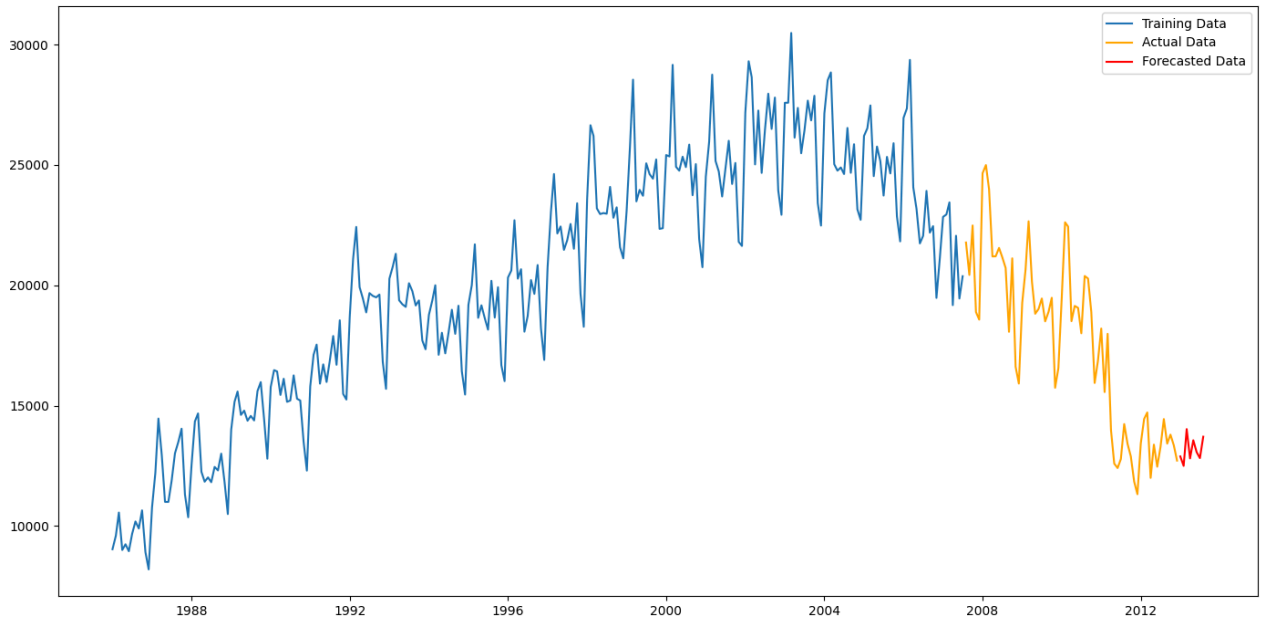



```
df_final=pd.DataFrame({'Date':pred_future.index, 'Total':pred_future.values})
```

```
df_final['Date'] = pd.to_datetime(df_final['Date'])
```

```
df_final.set_index('Date', inplace=True)
```

```
plt.figure(figsize=(14,7))
plt.plot(train_data_2.index,train_data_2['Total'],label='Training Data')
plt.plot(test_data_2.index,test_data_2['Total'], label='Actual Data', color='orange')
plt.plot(df_final.index ,df_final['Total'],label='Forecasted Data', color='red')

plt.legend()
plt.tight_layout()
plt.show()
```

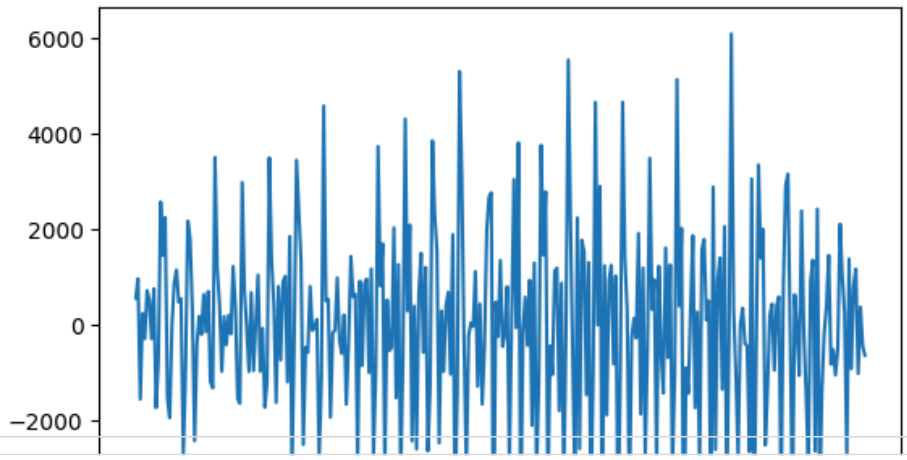


Difference the series then apply auto_arima

```
cat_df=difference(df_2['Total'])
```

```
plt.plot(cat_df)
```

```
[<matplotlib.lines.Line2D at 0x7c45a5e32ed0>]
```




```
train_size = int(len(df_2) * 0.80)
train_data_3, test_data_3 = cat_df[0:train_size], cat_df[train_size:]
```

```
model_4=ARIMA(train_data_2,order=(12,1,1))
model_4=model_4.fit()
model_4.summary()
```

SARIMAX Results

Dep. Variable:	Total	**No. Observations:**	259
Model:	ARIMA(12, 1, 1)	**Log Likelihood**	-2148.720
Date:	Mon, 15 Sep 2025	**AIC**	4325.441
Time:	17:32:11	**BIC**	4375.182
Sample:	01-01-1986 - 07-01-2007	**HQIC**	4345.442

Covariance Type: opg

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	0.0989	0.058	1.697	0.090	-0.015	0.213
ar.L2	-0.0783	0.051	-1.538	0.124	-0.178	0.021
ar.L3	-0.0946	0.056	-1.696	0.090	-0.204	0.015
ar.L4	-0.0034	0.055	-0.061	0.951	-0.111	0.104
ar.L5	0.0493	0.052	0.950	0.342	-0.052	0.151
ar.L6	-0.1073	0.047	-2.261	0.024	-0.200	-0.014
ar.L7	0.0884	0.058	1.525	0.127	-0.025	0.202
ar.L8	-0.0739	0.054	-1.359	0.174	-0.180	0.033
ar.L9	-0.0404	0.050	-0.816	0.415	-0.137	0.057
ar.L10	0.0129	0.051	0.253	0.800	-0.087	0.113
ar.L11	0.0191	0.053	0.359	0.720	-0.085	0.124
ar.L12	0.6726	0.046	14.488	0.000	0.582	0.764
ma.L1	-0.5203	0.079	-6.595	0.000	-0.675	-0.366
sigma2	1.032e+06	1.04e+05	9.963	0.000	8.29e+05	1.23e+06

Ljung-Box (L1) (Q):	0.03	**Jarque-Bera (JB):**	1.98
Prob(Q):	0.85	**Prob(JB):**	0.37
Heteroskedasticity (H):	1.57	**Skew:**	0.21
Prob(H) (two-sided):	0.04	**Kurtosis:**	2.89

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
pred_4=model_4.predict(start=start_2,end=end_2,type='levels')
```

```
rmse=sqrt(mean_squared_error(pred_4,test_data_2['Total']))
print(rmse)
```

```
3061.4209907593654
```

```
pred_4
```

	predicted_mean
2007-08-01	21587.944493
2007-09-01	20221.382687
2007-10-01	20621.587603
2007-11-01	17841.922752
2007-12-01	19634.275952
...	...
2012-08-01	18401.098633
2012-09-01	17767.420594
2012-10-01	17874.371953
2012-11-01	16831.329089
2012-12-01	17769.086414

65 rows × 1 columns

dtype: float64

```
import pandas as pd
from datetime import datetime
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima_model import ARMA
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from datetime import datetime
from datetime import timedelta
```

```
def parser(s):
    return datetime.strptime(s, '%Y-%m-%d')
```

```
def perform_adf_test(series):
    result = adfuller(series)
    print('ADF Statistic: %f' % result[0])
    print('p-value: %f' % result[1])
```

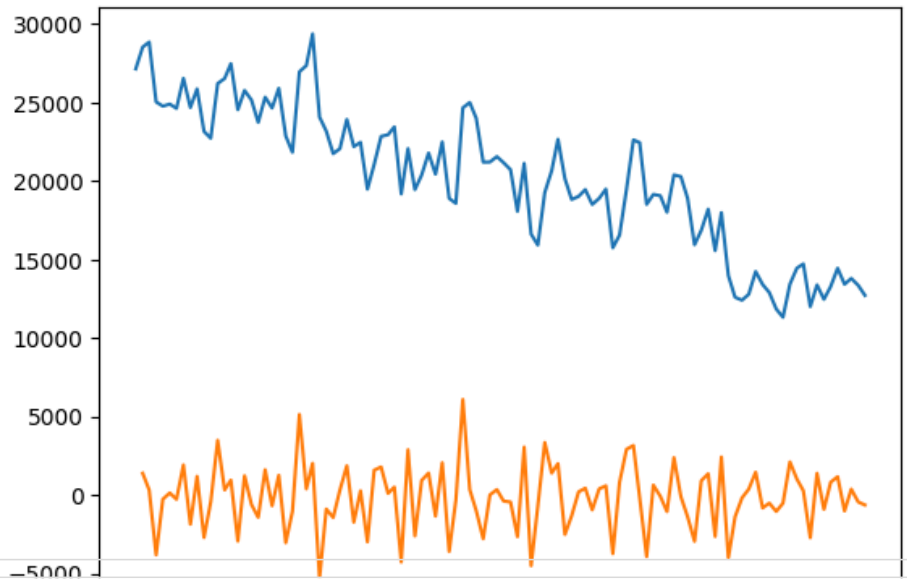
```
series = pd.read_csv('catfish.csv', parse_dates=[0], index_col=0, date_parser=parser)
```

```
series = series.asfreq(pd.infer_freq(series.index))
series = series.loc[datetime(2004,1,1):]
```

```
series_1 = series.diff().dropna()
```

```
plt.plot(series)
plt.plot(series_1)
```

```
[<matplotlib.lines.Line2D at 0x7c45a4e2cc20>]
```



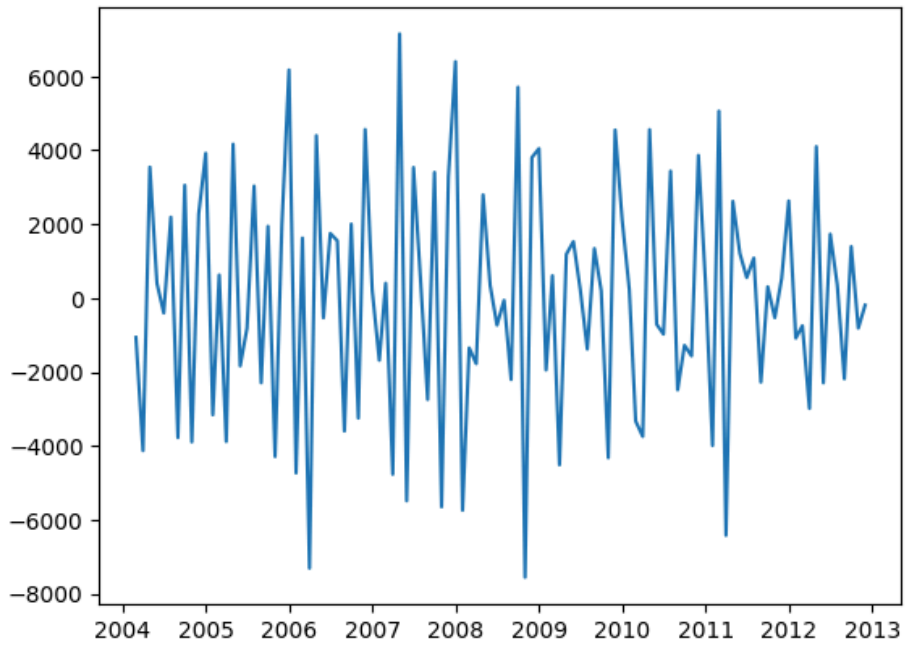

```
series_2 = series_1.diff().dropna()
```

```
perform_adf_test(series_2)
```

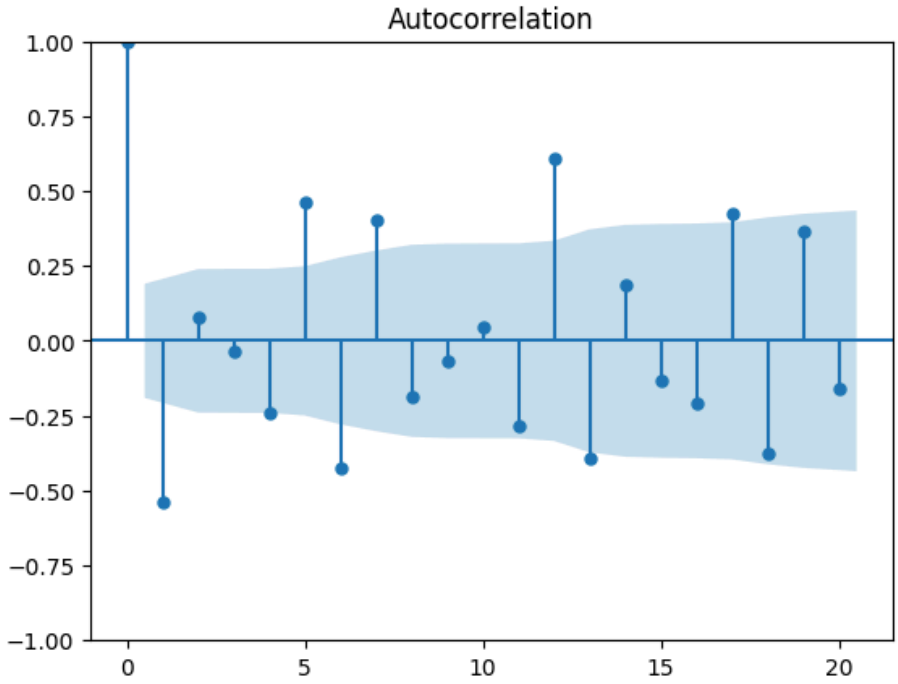
```
ADF Statistic: -7.162321
p-value: 0.000000
```

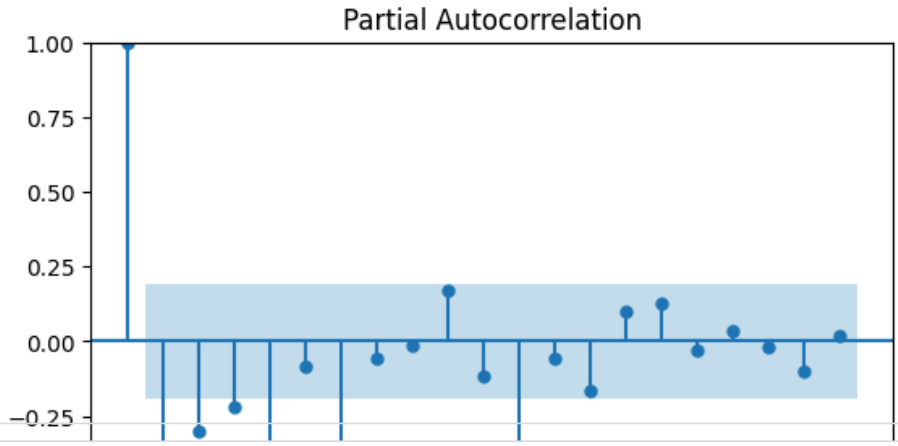
```
plt.plot(series_2)
```

```
[<matplotlib.lines.Line2D at 0x7c45a4c1dd00>]
```




```
plot_acf(series_2, lags=20)
plot_pacf(series_2, lags=20)
plt.show()
```






```
ar_orders = [1, 2, 3]
ma_orders = [5, 6, 7]
fitted_model_dict = {}

for ar_order in ar_orders:
    for ma_order in ma_orders:
        # Create ARMA(p,q) model
        arma_model = ARIMA(series_2, order=(ar_order, 0, ma_order))
        arma_model_fit = arma_model.fit()
        fitted_model_dict[(ar_order, ma_order)] = arma_model_fit
```

```
from statsmodels.tsa.arima.model import ARIMA
from itertools import product

p_values = range(1, 3)
q_values = range(1, 3)

fitted_model_dict = {}

for p, q in product(p_values, q_values):
    # Create ARMA(p,q) model
    arma_model = ARIMA(series_2, order=(p, 0, q))
    arma_model_fit = arma_model.fit()

    # Store AIC and BIC scores
    aic_score = arma_model_fit.aic
    bic_score = arma_model_fit.bic

    fitted_model_dict[(p, q)] = {'AIC': aic_score, 'BIC': bic_score}
```

```
# Print AIC and BIC scores for each combination
```

```
for (p, q), scores in fitted_model_dict.items():
    print(f'ARMA({p},{q}) - AIC: {scores["AIC"]}, BIC: {scores["BIC"]}')
```

```
ARMA(1,1) - AIC: 1925.101919524895, BIC: 1935.7556759013432
ARMA(1,2) - AIC: 1910.8157063931574, BIC: 1924.1329018637177
ARMA(2,1) - AIC: 1924.15595736913, BIC: 1937.4731528396903
ARMA(2,2) - AIC: 1921.777662911171, BIC: 1937.7582974758434
```

```
train_end = datetime(2011,12,1)
test_end = datetime(2012,12,1)

train_data = series_2[:train_end]
test_data = series_2[train_end + timedelta(days=1):test_end]
```

```
model = ARIMA(train_data, order=(2,1,2))
```

```
model_fit = model.fit()
```

```
print(model_fit.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                  Total   No. Observations:                   94
Model:                 ARIMA(2, 1, 2)   Log Likelihood                -864.804
Date:                Mon, 15 Sep 2025   AIC                           1739.608
Time:                        17:45:53   BIC                           1752.271
Sample:                    03-01-2004   HQIC                          1744.721
                         - 12-01-2011
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -1.3752      0.101    -13.656      0.000      -1.573      -1.178
ar.L2         -0.6232      0.095     -6.561      0.000      -0.809      -0.437
ma.L1         -0.1632      0.159     -1.025      0.305      -0.475       0.149
ma.L2         -0.8368      0.175     -4.789      0.000      -1.179      -0.494
sigma2      6.632e+06   3.76e-08   1.76e+14      0.000    6.63e+06    6.63e+06
===================================================================================
Ljung-Box (L1) (Q):                   0.23   Jarque-Bera (JB):                 0.11
Prob(Q):                              0.63   Prob(JB):                         0.95
Heteroskedasticity (H):               0.83   Skew:                             0.03
Prob(H) (two-sided):                  0.62   Kurtosis:                         3.16
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 2.88e+29. Standard errors ma
```

```
pred_start_date = test_data.index[0]
pred_end_date = test_data.index[-1]
```

```
pred_end_date
```

```
Timestamp('2012-12-01 00:00:00')
```

```
predictions = model_fit.predict(start=pred_start_date, end=pred_end_date)
residuals = test_data - predictions
```

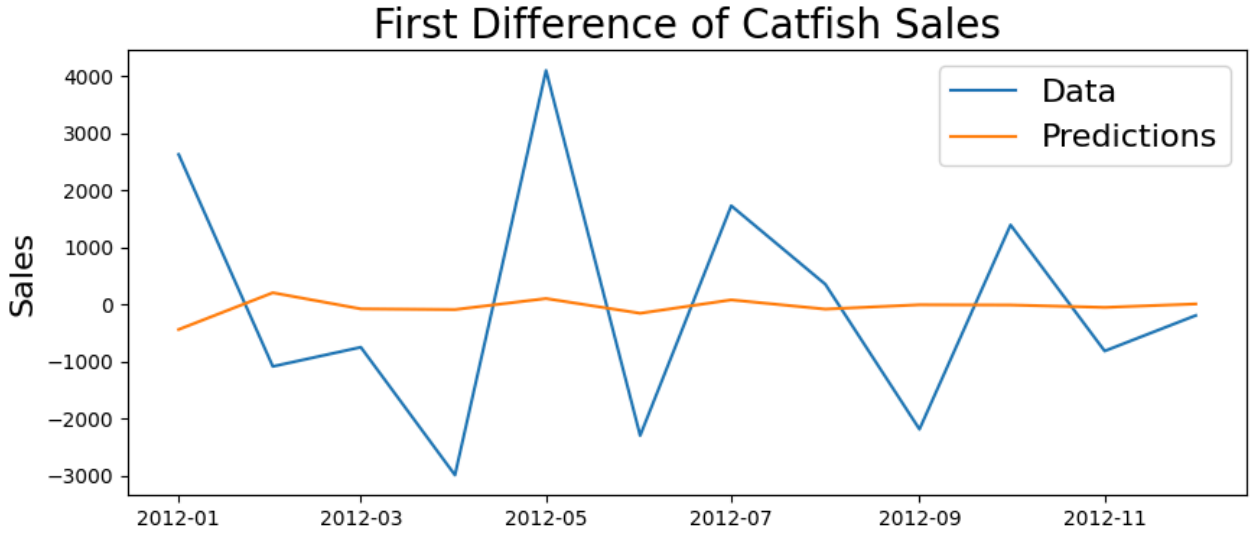
```
predictions
```

	predicted_mean
2012-01-01	-438.581944
2012-02-01	205.147882
2012-03-01	-77.747048
2012-04-01	-89.873018
2012-05-01	103.097069
2012-06-01	-154.716525
2012-07-01	79.570600
2012-08-01	-81.953986
2012-09-01	-5.830377
2012-10-01	-9.855730
2012-11-01	-51.758856
2012-12-01	8.374366

dtype: float64

```
plt.figure(figsize=(10,4))

plt.plot(test_data)
plt.plot(predictions)

plt.legend(('Data', 'Predictions'), fontsize=16)

plt.title('First Difference of Catfish Sales', fontsize=20)
plt.ylabel('Sales', fontsize=16)
```

```
Text(0, 0.5, 'Sales')
```




```
def invert_diff(last_actual, differences):
    inverted_values = []
    last_observation = last_actual
    for diff in differences:
        inverted_value = last_observation + diff
        inverted_values.append(inverted_value)
        last_observation = inverted_value
    return inverted_values
```

```
actual_values = pd.Series([100, 105, 110, 115, 120])

# Sample differenced data (assuming first order differencing)
differenced_data = pd.Series([5, 5, 5, 5])

# Sample model forecast (assuming one-step-ahead forecast)
```

```
actual_values = pd.Series([100, 105, 110, 115, 120])
actual_values.iloc[-1]
```

```
np.int64(120)
```

```
invert_diff(actual_values.iloc[-1],differenced_data)
```

```
[np.int64(125), np.int64(130), np.int64(135), np.int64(140)]
```

```
series = pd.read_csv('city_day.csv')
series_delhi = series.loc[series['City'] == 'Delhi']
ts_delhi = series_delhi[['Date','AQI']]
#converting 'Date' column to type 'datetime' so that indexing can happen later
ts_delhi['Date'] = pd.to_datetime(ts_delhi['Date'])
```

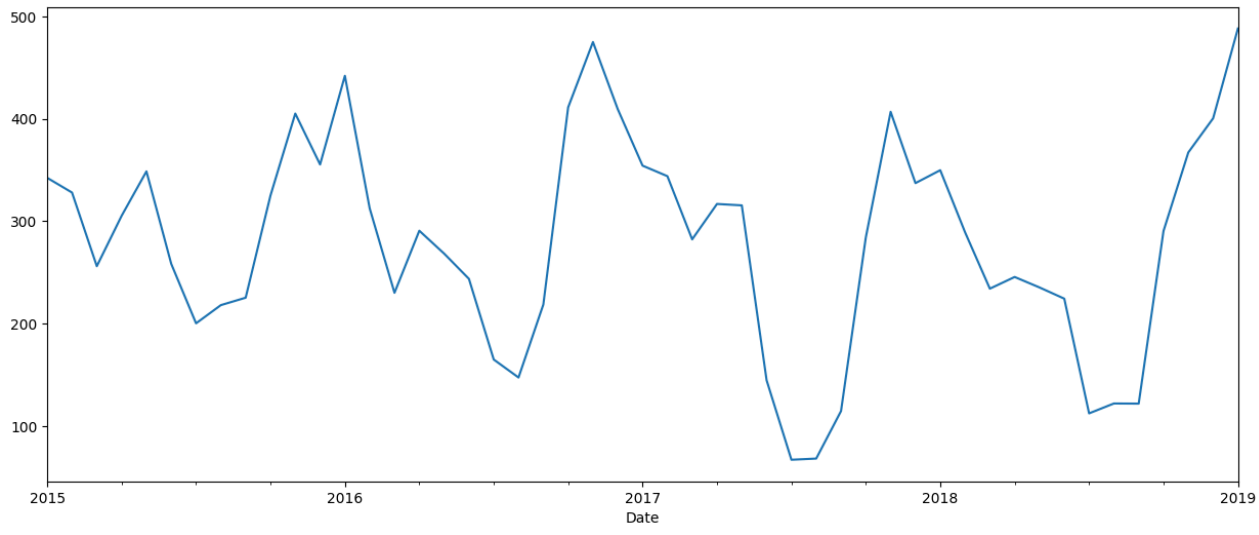
```
ts_delhi.isnull().sum()
ts_delhi = ts_delhi.dropna()
ts_delhi.isnull().sum()
```

	0
Date	0
AQI	0

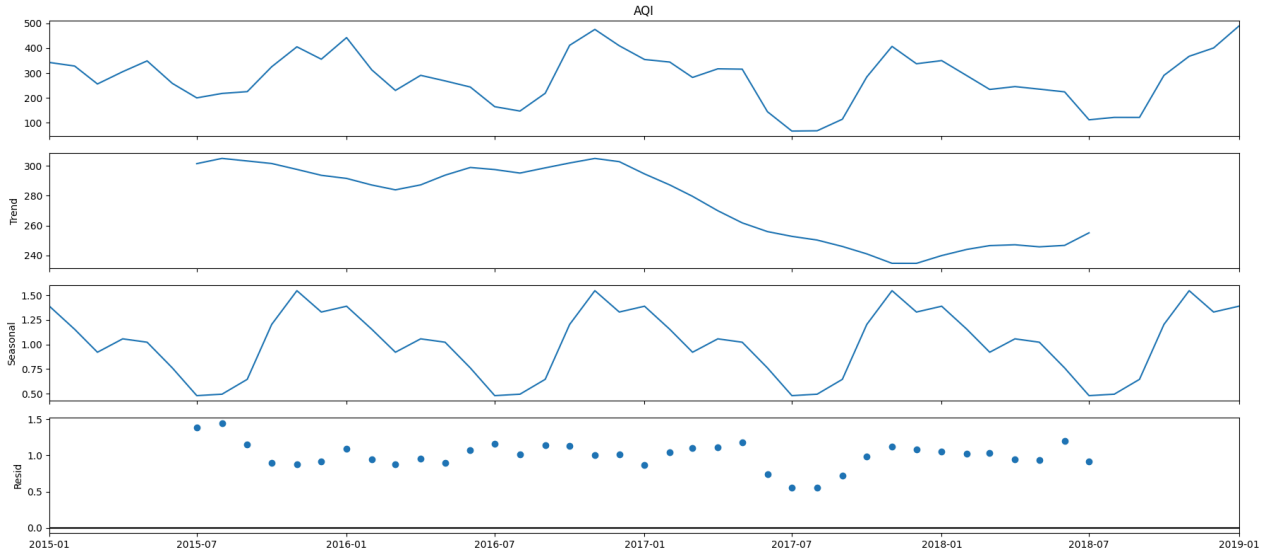
dtype: int64

```
ts_delhi = ts_delhi.set_index('Date')
```

```
ts_month_avg = ts_delhi['AQI'].resample('MS').mean()
ts_month_avg.plot(figsize = (15, 6))
plt.show()
```



```
from pylab import rcParams
rcParams['figure.figsize'] = 18, 8
decomposition = sm.tsa.seasonal_decompose(ts_month_avg, model='multiplicative')
fig = decomposition.plot()
plt.show()
```




```
adf_test = adfuller(ts_month_avg)

print('ADF Statistic: %f' % adf_test[0])
print('Critical Values @ 0.05: %.2f' % adf_test[4]['5%'])
print('p-value: %f' % adf_test[1])
```

```
ADF Statistic: -1.208958
Critical Values @ 0.05: -2.94
p-value: 0.669731
```

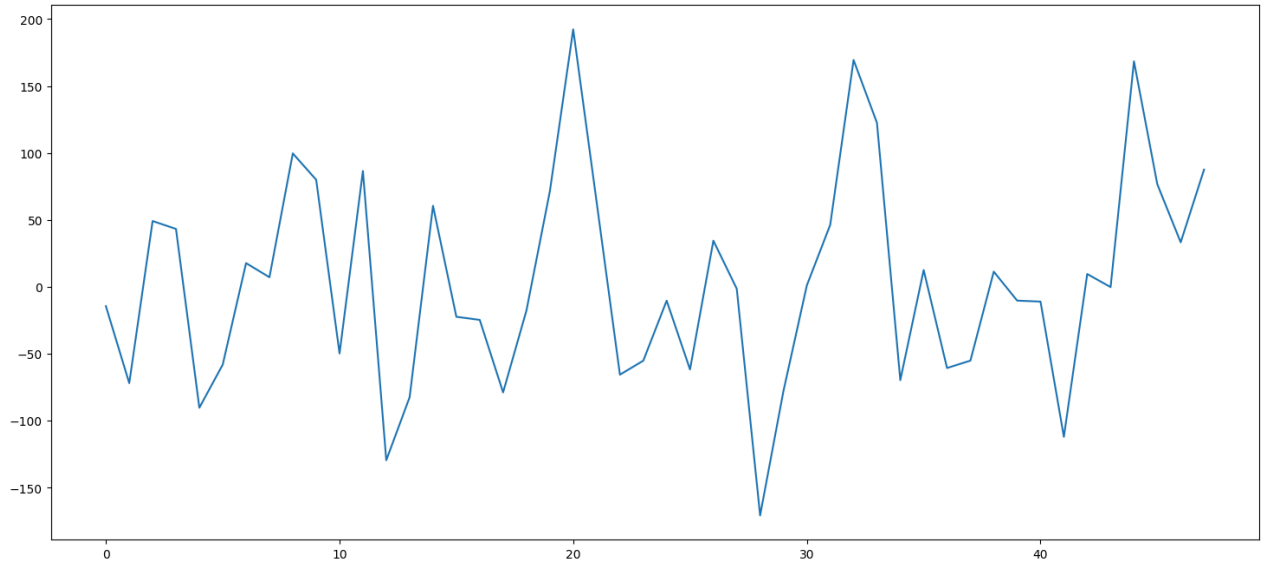
```
def difference(dataset, interval=1):
 diff = list()
 for i in range(interval, len(dataset)):
     value = dataset[i] - dataset[i - interval]
     diff.append(value)
 return Series(diff)
```

```
ts_t_adj=difference(ts_month_avg)
```

```
ts_t_adj=ts_t_adj.dropna()
```

```
ts_t_adj.plot()
```

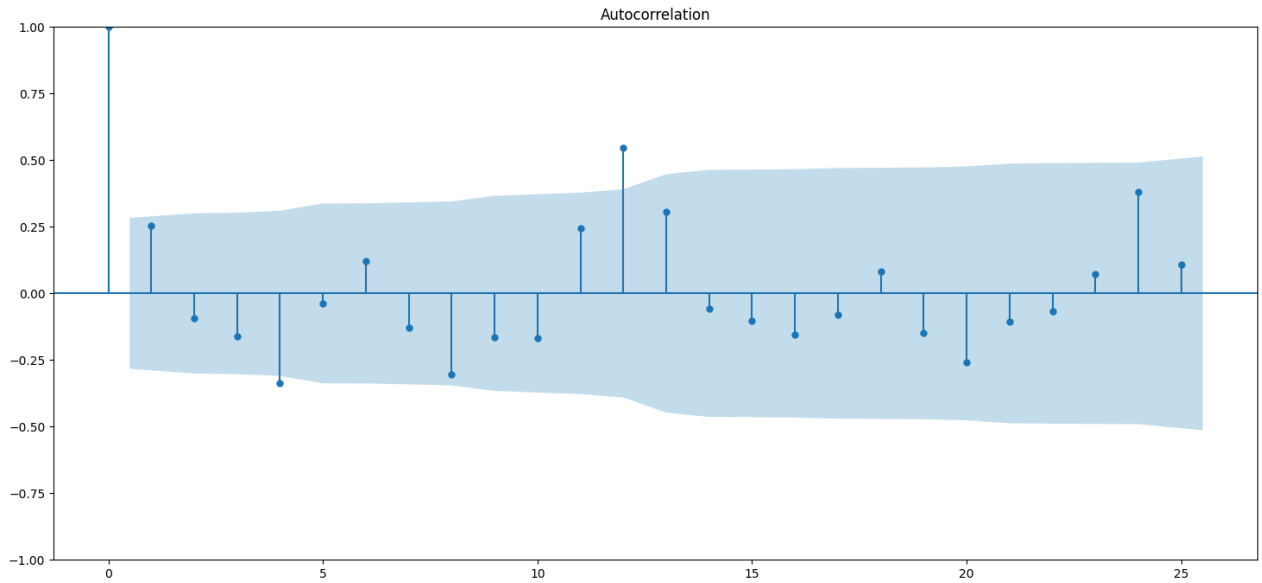
```
<Axes: >
```

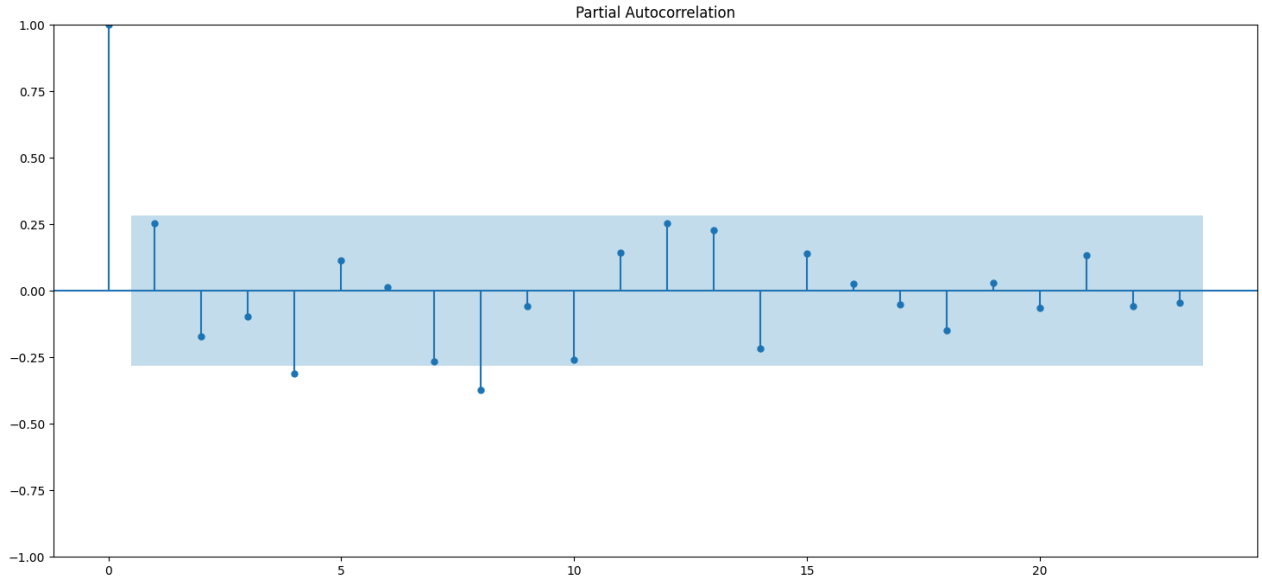



```
adf_test = adfuller(ts_t_adj)

print('ADF Statistic: %f' % adf_test[0])
print('Critical Values @ 0.05: %.2f' % adf_test[4]['5%'])
print('p-value: %f' % adf_test[1])
```

```
ADF Statistic: -5.899664
Critical Values @ 0.05: -2.94
p-value: 0.000000
```

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_acf(ts_t_adj,lags=25)
plot_pacf(ts_t_adj,lags=23)
plt.show()
```

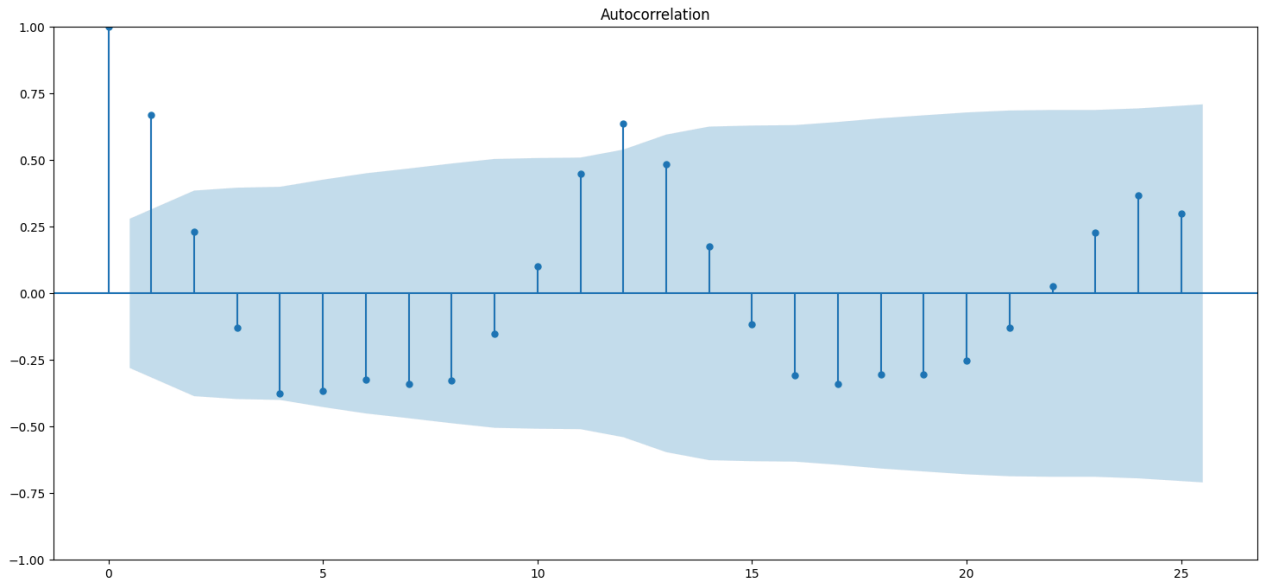


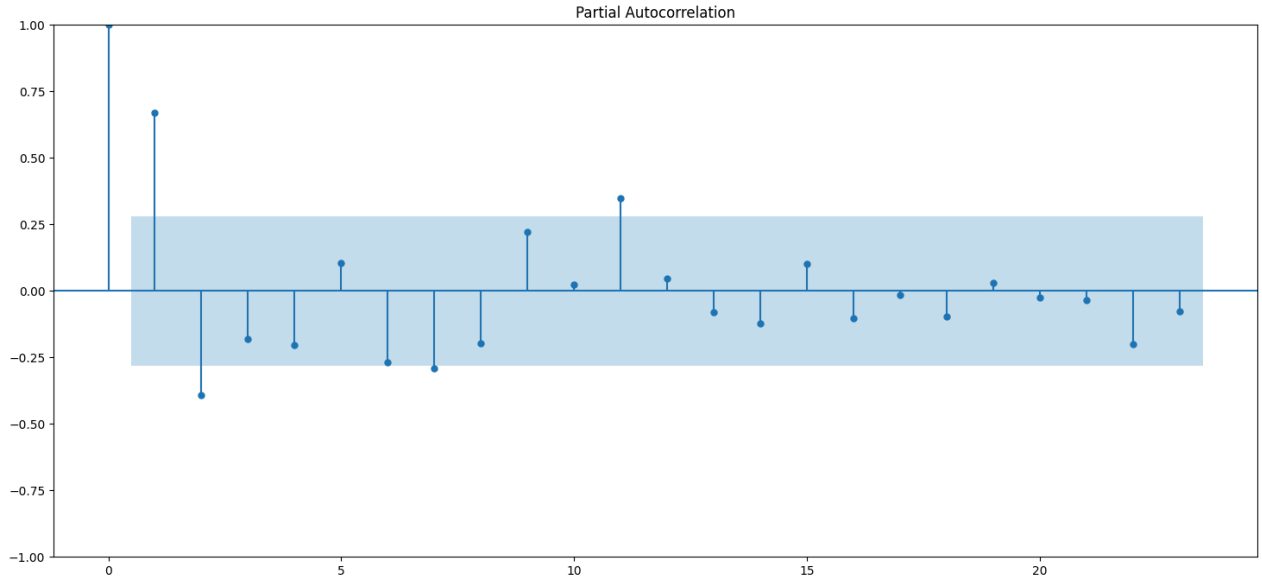



```
train = ts_month_avg[:'2017-12-01']
test = ts_month_avg['2018-01-01':]
print('Train Timeseries Range => ', train.index.min(), ' - ' , train.index.max())
print('Test Timeseries Range => ', test.index.min(), ' - ' , test.index.max())
```

```
Train Timeseries Range =>  2015-01-01 00:00:00  -  2017-12-01 00:00:00
Test Timeseries Range =>  2018-01-01 00:00:00  -  2019-01-01 00:00:00
```

```
plot_acf(ts_month_avg,lags=25)
plot_pacf(ts_month_avg,lags=23)
plt.show()
```






```
from statsmodels.tsa.statespace.sarimax import SARIMAX

sarima = SARIMAX(train,order=(1,1,1), seasonal_order=(1,0,1,12),
                                 enforce_stationarity=False, enforce_invertibility=False,)
```

```
fitted = sarima.fit()
print(fitted.summary())
```

```
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                                 AQI   No. Observations:                   49
Model:             SARIMAX(1, 1, 1)x(1, 0, 1, 12)   Log Likelihood               -1168.236
Date:                            Mon, 15 Sep 2025   AIC                           2346.472
Time:                                    17:54:53   BIC                           2354.104
Sample:                                01-01-2015   HQIC                          2349.074
                                     - 01-01-2019
Covariance Type:                             opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.0114     44.790      0.000      1.000     -87.776      87.798
ma.L1          3.3003      1.224      2.696      0.007       0.901       5.700
ar.S.L12       0.3084   1.02e+04   3.03e-05      1.000   -1.99e+04    1.99e+04
ma.S.L12    1.975e+13   3.84e-08   5.15e+20      0.000    1.97e+13    1.97e+13
sigma2         68.7715     40.690      1.690      0.091     -10.980     148.523
==============================================================================
```

```
Ljung-Box (L1) (Q):                   2.81   Jarque-Bera (JB):                27.70
Prob(Q):                              0.09   Prob(JB):                         0.00
Heteroskedasticity (H):               0.00   Skew:                            -0.77
Prob(H) (two-sided):                  0.00   Kurtosis:                         7.15
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number    inf. Standard errors may
```

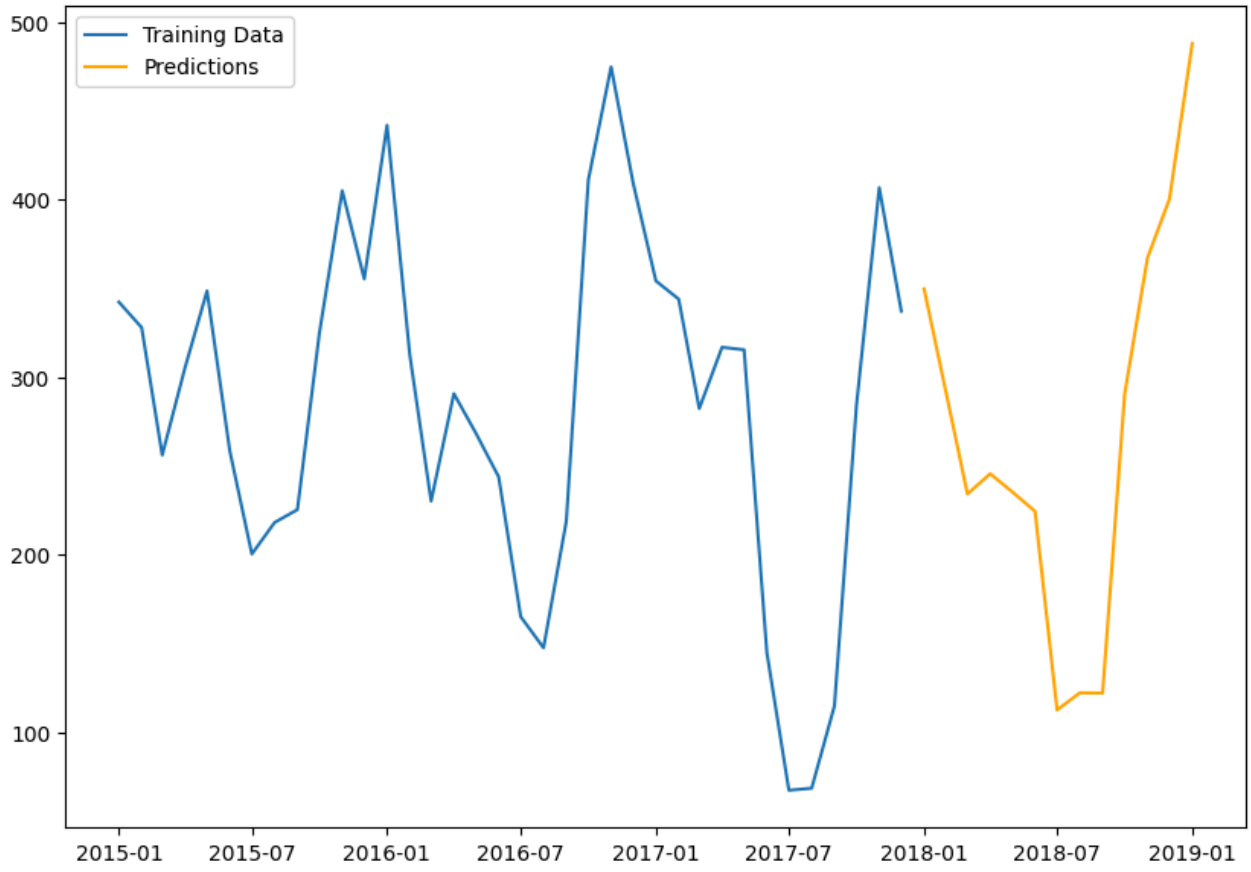
```
pred_test = fitted.predict(start=test.index.min(), end=test.index.max())
```

```
pred_test
```

Date	predicted_mean
2018-01-01	-2.524341e+06
2018-02-01	7.653312e+05
2018-03-01	-2.315392e+05
2018-04-01	7.047198e+04
2018-05-01	-2.103358e+04
2018-06-01	6.627078e+03
2018-07-01	-1.739124e+03
2018-08-01	6.727927e+02
2018-09-01	-3.040124e+01
2018-10-01	2.202063e+02
2018-11-01	3.508195e+02
2018-12-01	3.509616e+02
2019-01-01	4.198754e+02

dtype: float64

```
plt.figure(figsize=(10,7))
plt.plot(train.index,train,label='Training Data')
plt.plot(test.index,test, label='Predictions', color='orange')
plt.legend()
plt.show()
```


Training Data
Predictions
500
400
300
200
100
2015-01
2015-07
2016-01
2016-07
2017-01
2017-07
2018-01
2018-07
2019-01

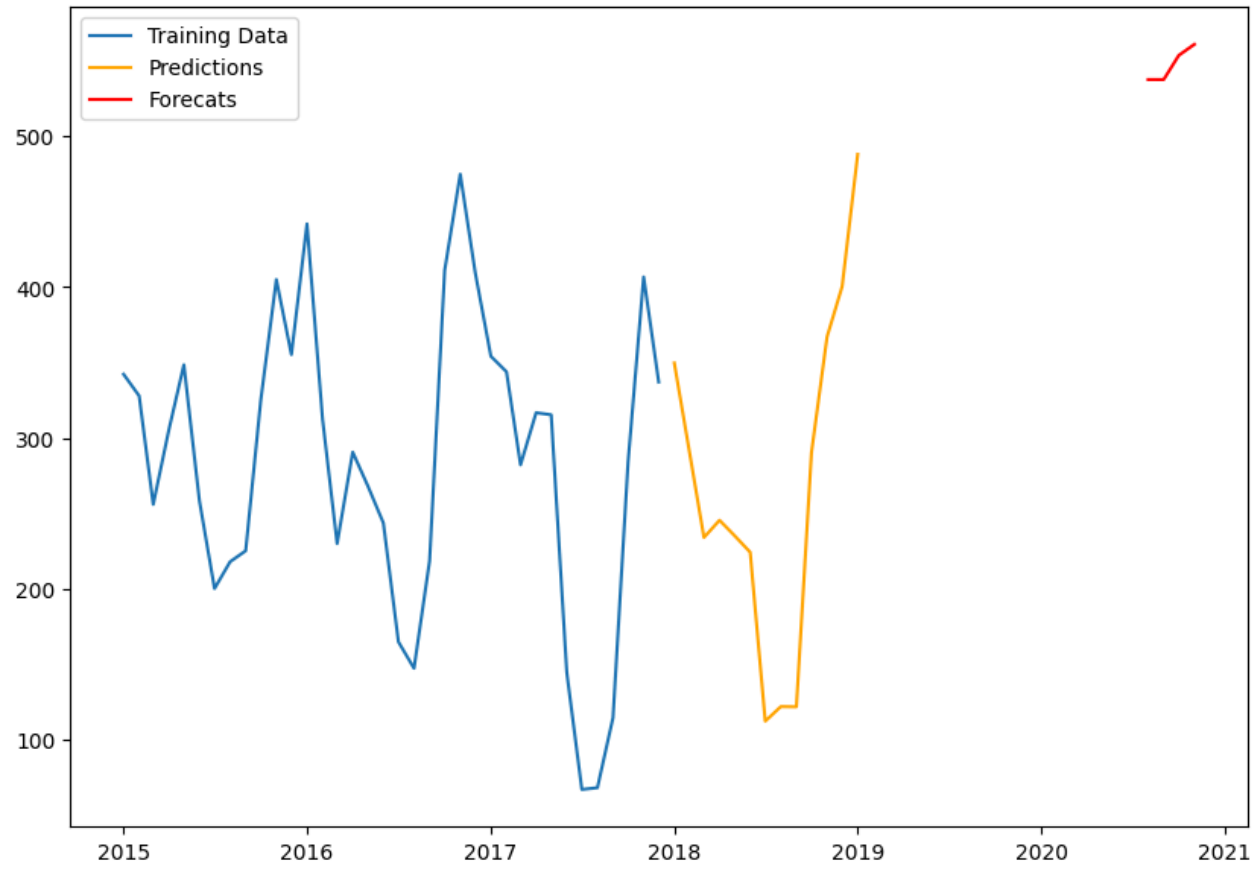
```
ts_month_avg
```

Date	AQI
2015-01-01	342.290323
2015-02-01	327.928571
2015-03-01	256.064516
2015-04-01	305.266667
2015-05-01	348.580645
2015-06-01	258.333333
2015-07-01	200.290323
2015-08-01	218.064516
2015-09-01	225.300000

```
pred_future = fitted.get_prediction(start=pd.to_datetime('2020-08-01'),end=pd.to_datetime('2020-11-01
```

2015-11-01 405.033333

```
plt.figure(figsize=(10,7))
plt.plot(train.index,train,label='Training Data')
plt.plot(test.index,test, label='Predictions', color='orange')
plt.plot(pred_future.predicted_mean.index,pred_future.predicted_mean.values, label='Forecats', color=
plt.legend()
plt.show()
```



2017-10-01 284.161290

2017-11-01 406.733333

```
from sklearn.metrics import mean_absolute_error
mae=mean_absolute_error(pred_test,test)
print(mae)
```

2018-02-01 289.178571

2018-03-01 234.129032

```
test.mean()
```

```
np.float64(267.7355724920241)
```

```
import statsmodels.api as sm

# Define the order and seasonal order for the SARIMAX model
p = 1
d = 1
q = 1
P = 1
D = 0
Q = 1

sarima_model = sm.tsa.statespace.SARIMAX(
    train,
    order=(p, d, q),          # e.g. (1,1,1)
    seasonal_order=(P, D, Q, 12),  # e.g. (1,1,1,12)
    enforce_stationarity=False,
    enforce_invertibility=False
).fit()

print(sarima_model.summary())
```

```
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                                AQI   No. Observations:                   36
Model:             SARIMAX(1, 1, 1)x(1, 0, 1, 12)   Log Likelihood                -113.222
Date:                            Mon, 15 Sep 2025   AIC                            236.444
Time:                                    17:59:14   BIC                            241.667
Sample:                                01-01-2015   HQIC                           237.578
                                     - 12-01-2017
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.5338      0.346      1.544      0.123      -0.144       1.212
ma.L1         -1.0000   1738.780     -0.001      1.000   -3408.946    3406.946
ar.S.L12       1.0559      0.170      6.223      0.000       0.723       1.389
ma.S.L12      -1.0000   1738.777     -0.001      1.000   -3408.940    3406.939
sigma2      1416.5133      1.212   1169.037      0.000    1414.138    1418.888
===================================================================================
Ljung-Box (L1) (Q):                   0.07   Jarque-Bera (JB):                 0.63
Prob(Q):                              0.79   Prob(JB):                         0.73
Heteroskedasticity (H):               0.85   Skew:                            -0.42
Prob(H) (two-sided):                  0.83   Kurtosis:                         3.03
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 3.3e+22. Standard errors may
```

```
print(sarima_model.summary())
```

```
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                                AQI   No. Observations:                   36
Model:             SARIMAX(1, 1, 1)x(1, 0, 1, 12)   Log Likelihood                -113.222
Date:                            Mon, 15 Sep 2025   AIC                            236.444
Time:                                    17:59:18   BIC                            241.667
Sample:                                01-01-2015   HQIC                           237.578
                                     - 12-01-2017
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.5338      0.346      1.544      0.123      -0.144       1.212
ma.L1         -1.0000   1738.780     -0.001      1.000   -3408.946    3406.946
ar.S.L12       1.0559      0.170      6.223      0.000       0.723       1.389
```

```
ma.S.L12      -1.0000   1738.777     -0.001      1.000   -3408.940    3406.939
sigma2      1416.5133      1.212   1169.037      0.000    1414.138    1418.888
===================================================================================
Ljung-Box (L1) (Q):                   0.07   Jarque-Bera (JB):                 0.63
Prob(Q):                              0.79   Prob(JB):                         0.73
Heteroskedasticity (H):               0.85   Skew:                            -0.42
Prob(H) (two-sided):                  0.83   Kurtosis:                         3.03
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 3.3e+22. Standard errors may
```

```
auto=SARIMAX(train,order=(1,0,1), seasonal_order=(2,0,0,12),
                                enforce_stationarity=False, enforce_invertibility=False,)
```

```
fitted_2 = auto.fit()
```

```
pred_test_new = fitted_2.predict(start=test.index.min(), end=test.index.max())
```

```
pred_test_new
```

	predicted_mean
2018-01-01	436.602482
2018-02-01	280.442971
2018-03-01	176.792540
2018-04-01	256.307708
2018-05-01	231.400286
2018-06-01	187.695162
2018-07-01	86.729709
2018-08-01	67.731703
2018-09-01	161.187931
2018-10-01	413.739640
2018-11-01	505.134128
2018-12-01	420.823845
2019-01-01	365.121578

dtype: float64

```
from sklearn.metrics import mean_absolute_error
mae=mean_absolute_error(pred_test_new,test)
print(mae)
```

```
56.021598875372966
```

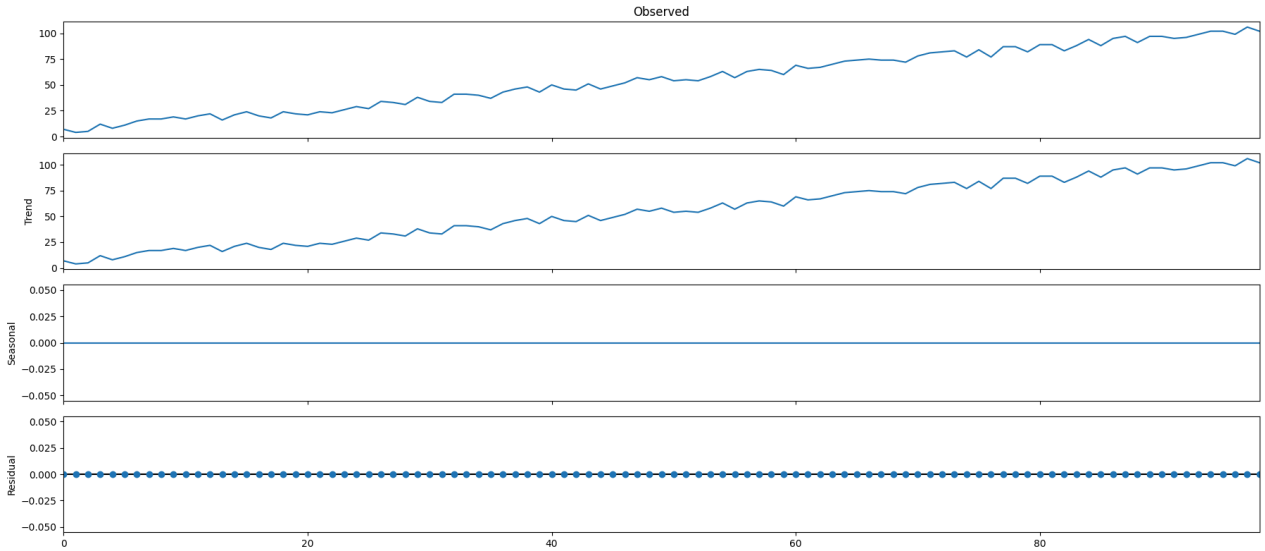
Introduction and Decomposing

```
import pandas as pd
import numpy as np
from statsmodels.tsa.seasonal import seasonal_decompose
from random import randrange
from pandas import Series
```

```
from matplotlib import pyplot
import matplotlib.pyplot as plt
```

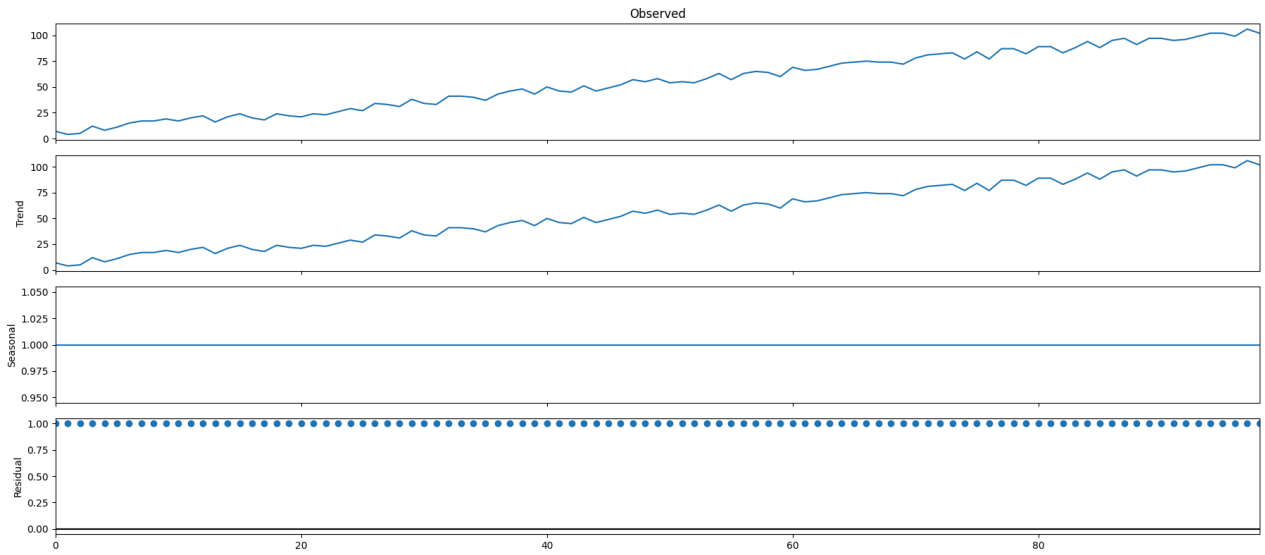
```
series = [i+randrange(10) for i in range(1,100)]
```

```
result = seasonal_decompose(series, model='additive', period=1)
result.plot()
pyplot.show()
```



Multiplicative

```
series_2 = [i**2.0 for i in range(1,100)]
result_2 = seasonal_decompose(series, model='multiplicative', period=1)
result_2.plot()
pyplot.show()
```



Example

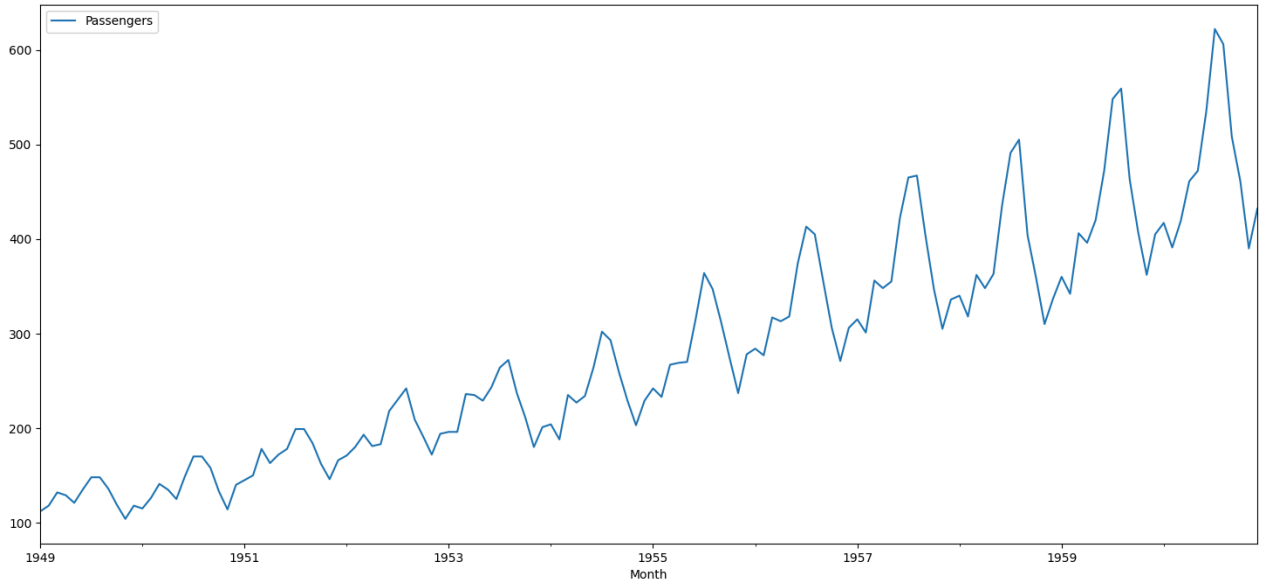
```
df=pd.read_csv('airline-passengers_data.csv')

df.head()
```

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

Next steps: Generate code with df | View recommended plots | New interactive sheet

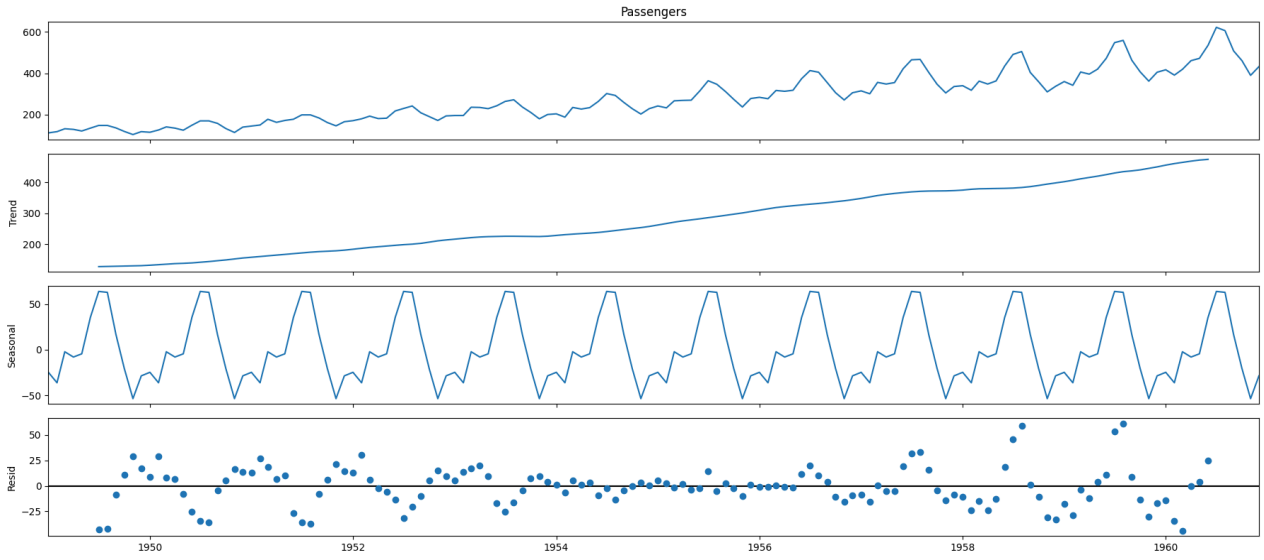
```
df.set_index('Month',inplace=True)
df.index=pd.to_datetime(df.index)
df.dropna(inplace=True)
df.plot()
```

```
<Axes: xlabel='Month'>
```



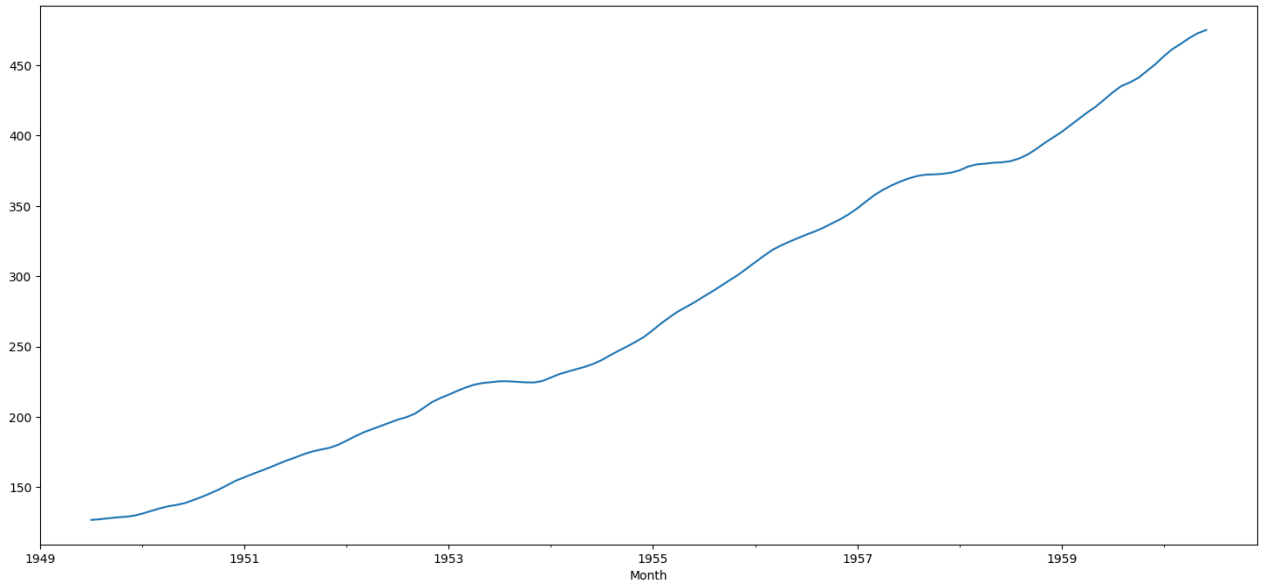

```
result_air=seasonal_decompose(df['Passengers'], model='additive', period=12)
```

```
result_air.plot()
pyplot.show()
```




```
result_air.trend.plot()
```

```
<Axes: xlabel='Month'>
```




```
result_air.seasonal.plot()
```